

Designing system commands

Chris Richardson

Microservice architecture consultant and trainer

Founder of the original CloudFoundry.com

Author of POJOs in Action and Microservices Patterns

Founder of Eventuate.io

 @crichardson

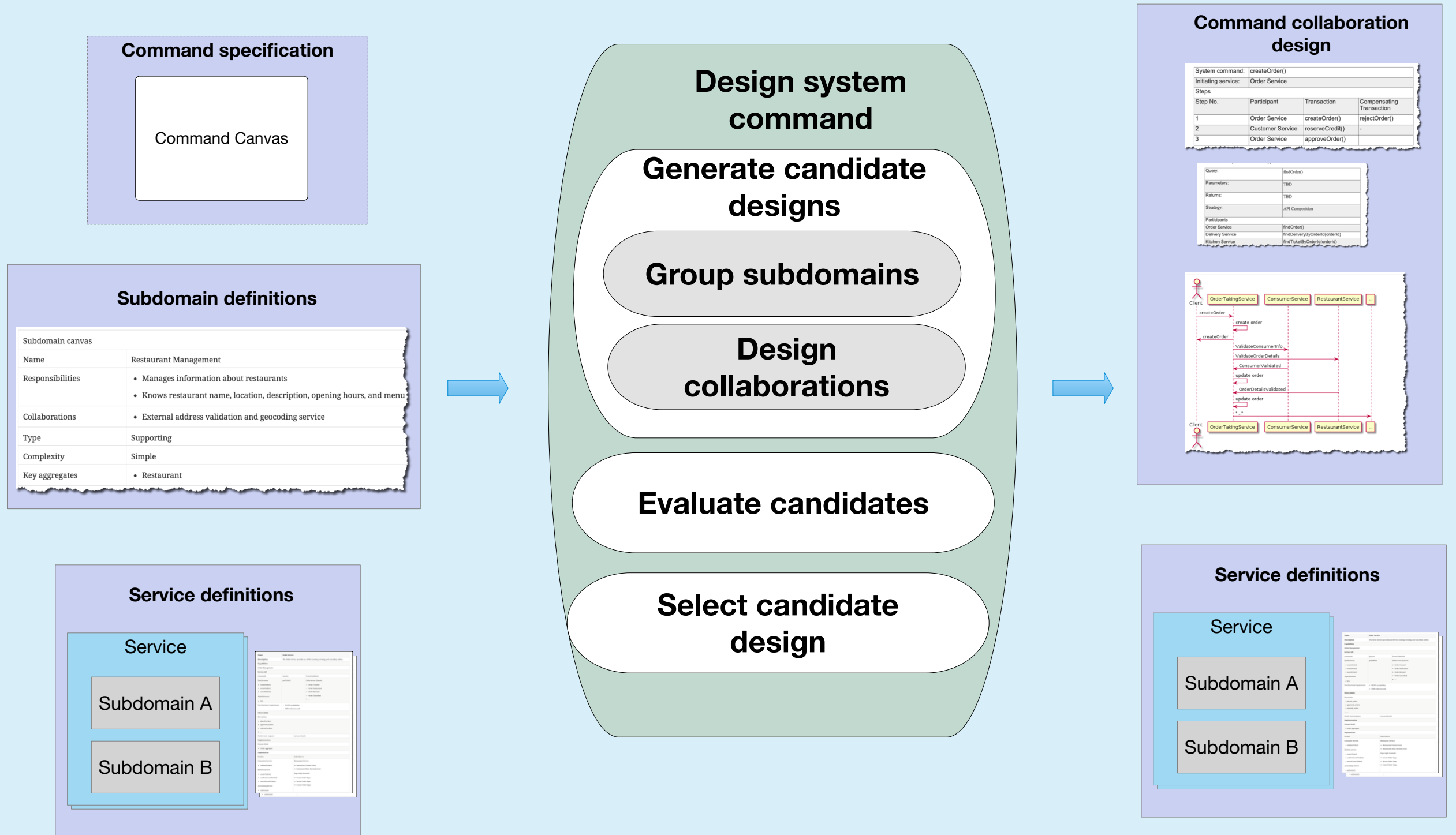
chris@chrisrichardson.net

<http://adopt.microservices.io>

Agenda

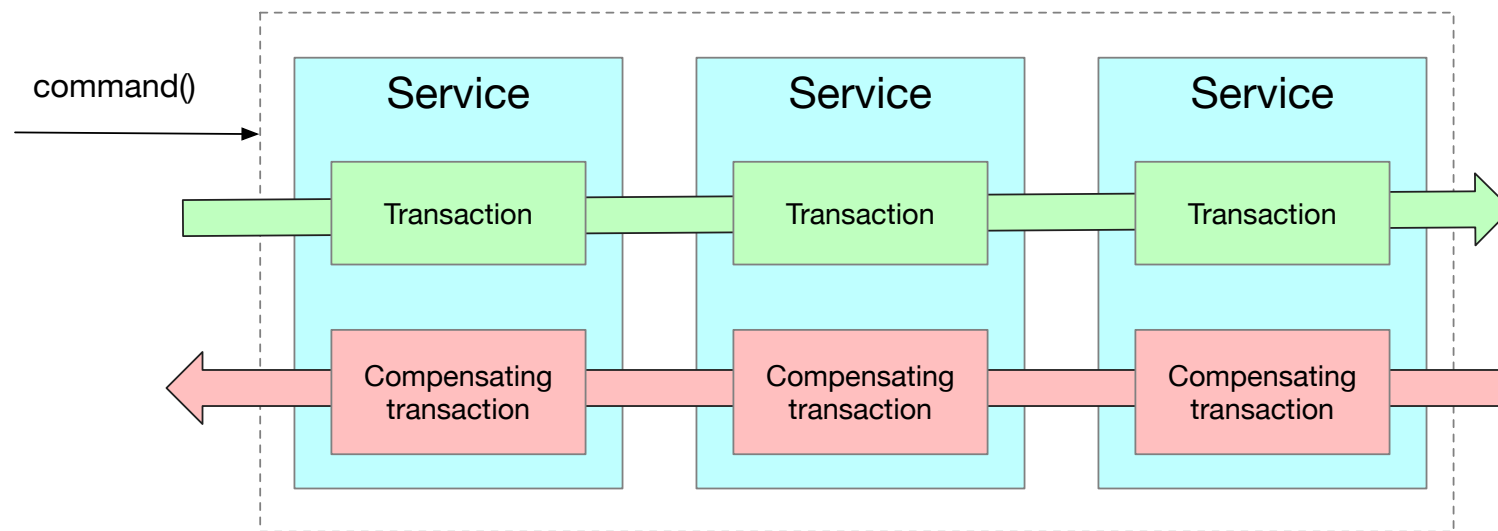
- Overview of designing commands
- The Saga pattern
- Designing sagas
- The Command-side replica pattern

Designing system commands

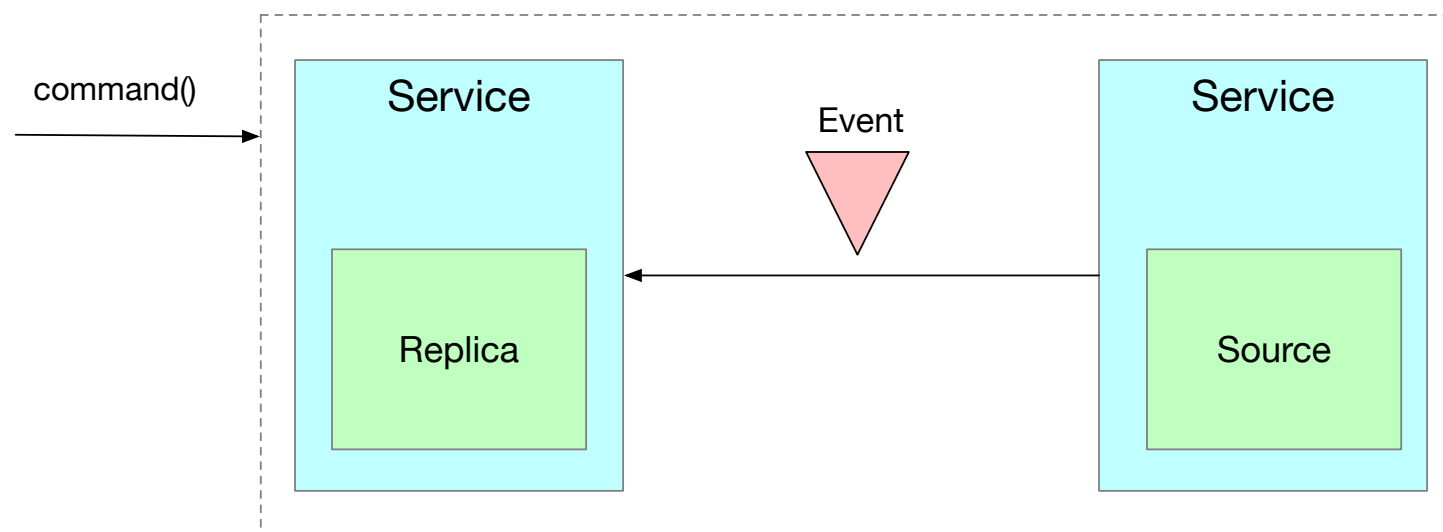


Command patterns

Saga



Command-side replica



Agenda

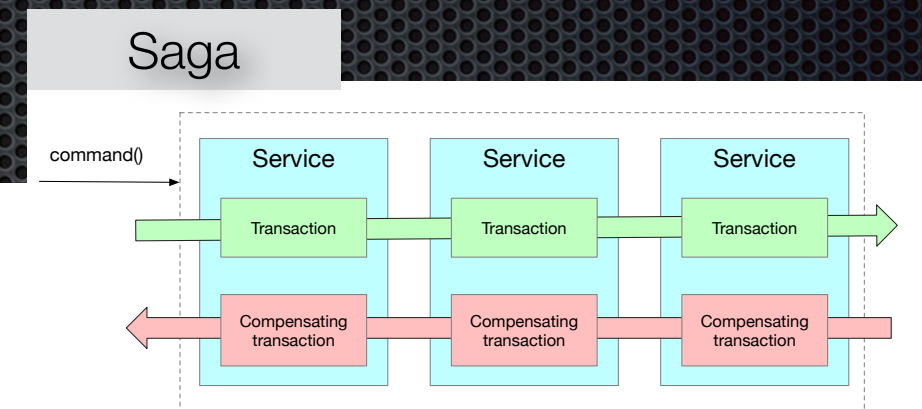
- Overview of designing commands
- The Saga pattern
- Designing sagas
- The Command-side replica pattern

Let's imagine that you are implementing the createOrder() command...

Service	Operation	Potential failure
Order Service	Create Order	Invalid quantities
Restaurant Service	Validate and price menu items (Order subtotal)	Invalid menu items
Customer Service	Reserve credit	Insufficient credit
Kitchen Service	Create Ticket	Out of stock ingredient

From a 1987 paper

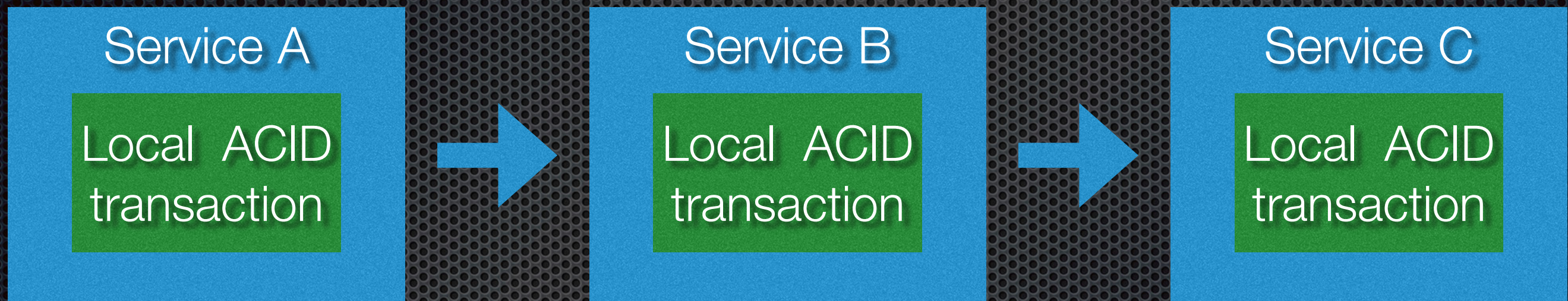
SAGAS



Hector Garcia-Molina
Kenneth Salem

Department of Computer Science
Princeton University
Princeton, N J 08544

Saga = sequence of local transactions



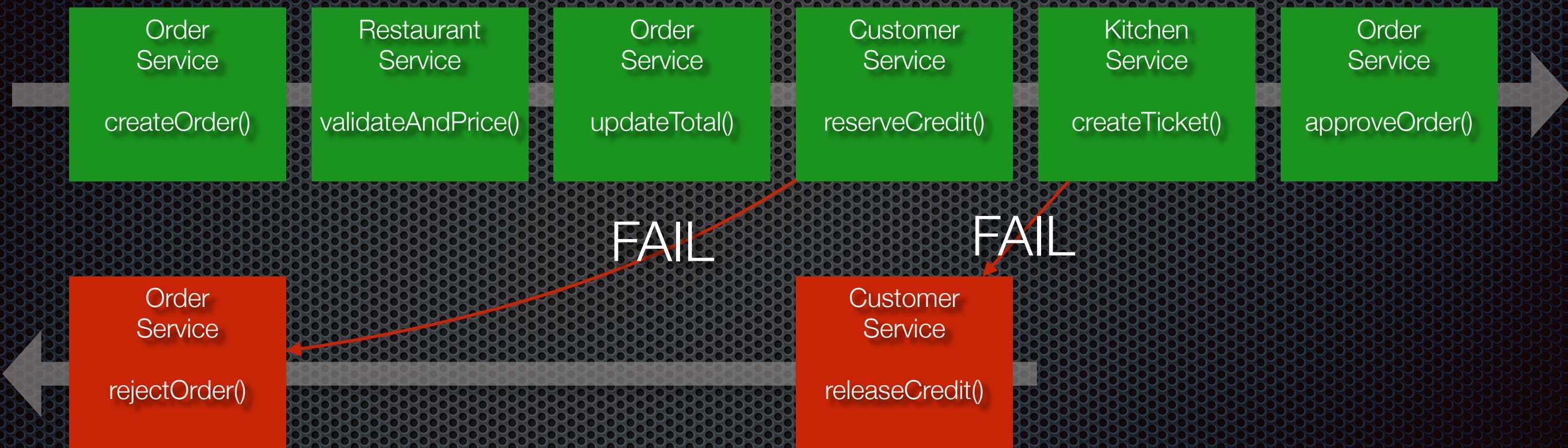
The CreateOrderSaga

Step	Participant	Transaction	Compensating transaction	Undo
1	Order Service	createOrder() Lock	rejectOrder() Unlock	
2	Restaurant Service	validateAndPrice()	-	
3	Order Service	updateSubtotal()	-	
4	Customer Service	reserveCredit()	releaseCredit()	
5	Kitchen Service	createTicket()		Can fail!
6	Order Service	approveOrder() Unlock		Can fail!

Need to rollback

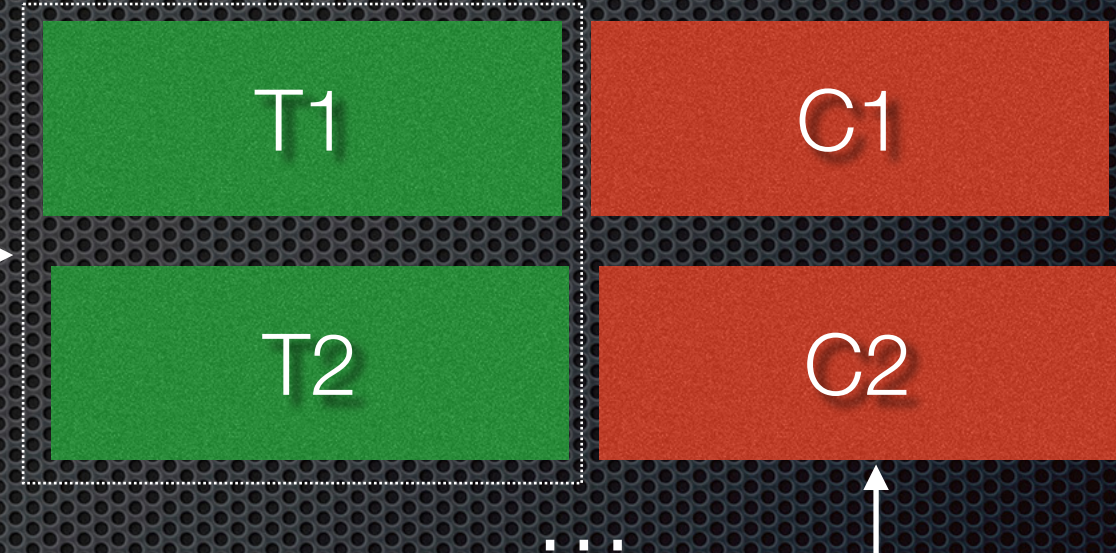
Semantic lock countermeasure = Semantic ACID

Execution of Create Order Saga

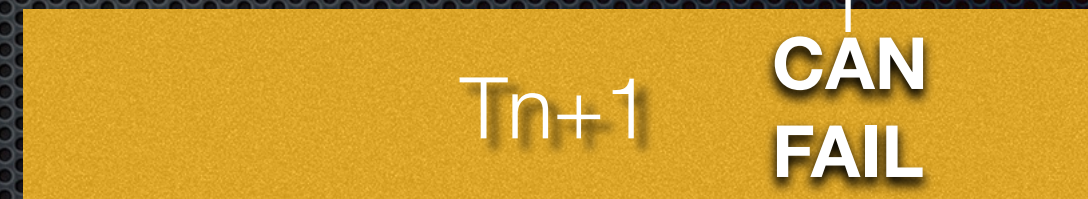


Saga structure: 3 phases

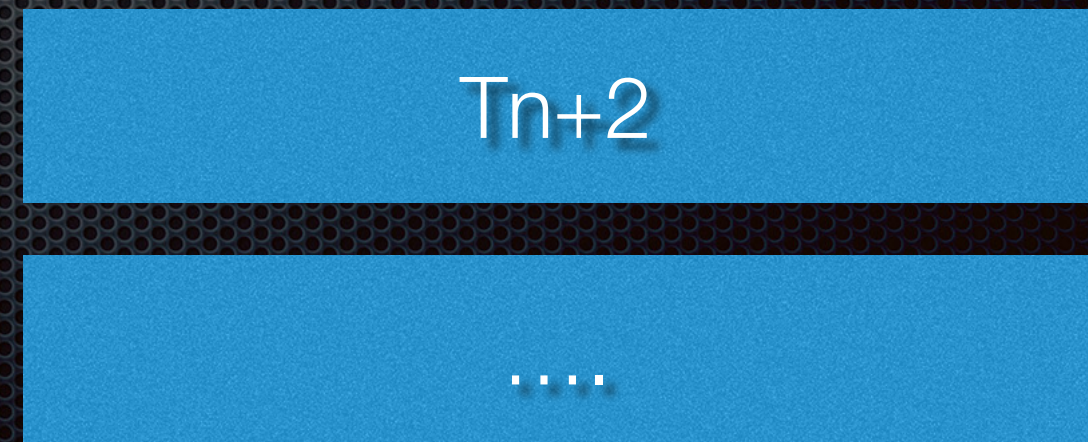
Compensatable transactions:



Pivot transaction = GO/NO GO



Retriable transactions that can't fail and don't need compensation



Great for actions that cannot be undone

3 phase saga: CreateOrderSaga

Step	Participant	Transaction	Compensating transaction	
1	Order Service	createOrder()	rejectOrder()	Compensatable
2	Restaurant Service	validateAndPrice()	-	
3	Order Service	updateSubtotal()	-	
4	Customer Service	reserveCredit()	releaseCredit()	Pivot
5	Kitchen Service	createTicket()		
6	Order Service	approveOrder()		Retriable

Not the best names but

SOFTWARE—PRACTICE AND EXPERIENCE, VOL. 28(1), 77–98 (JANUARY 1998)

Semantic ACID Properties in Multidatabases Using Remote Procedure Calls and Update Propagations

lars frank¹ and torben u. zahle²

<http://bit.ly/semantic-acid-ctm> - paywall 😓

About the Pivot transaction

- Pivot transaction: last step that can fail
 - A Saga is a sequence of steps => one of them must be the last one to fail = pivot transaction!
- None of the following steps can fail => Pivot transaction does not need a compensating transaction
- Useful for transactions that can fail but
 - cannot be undone => invoking external application that doesn't support undo
 - are inconvenient to undo => reversing a credit card charge

About retrievable transactions

- A retrievable transactions is guaranteed to succeed
- Good for operations that
 - cannot fail and cannot be undone, e.g. sending an email
 - are complex to undo

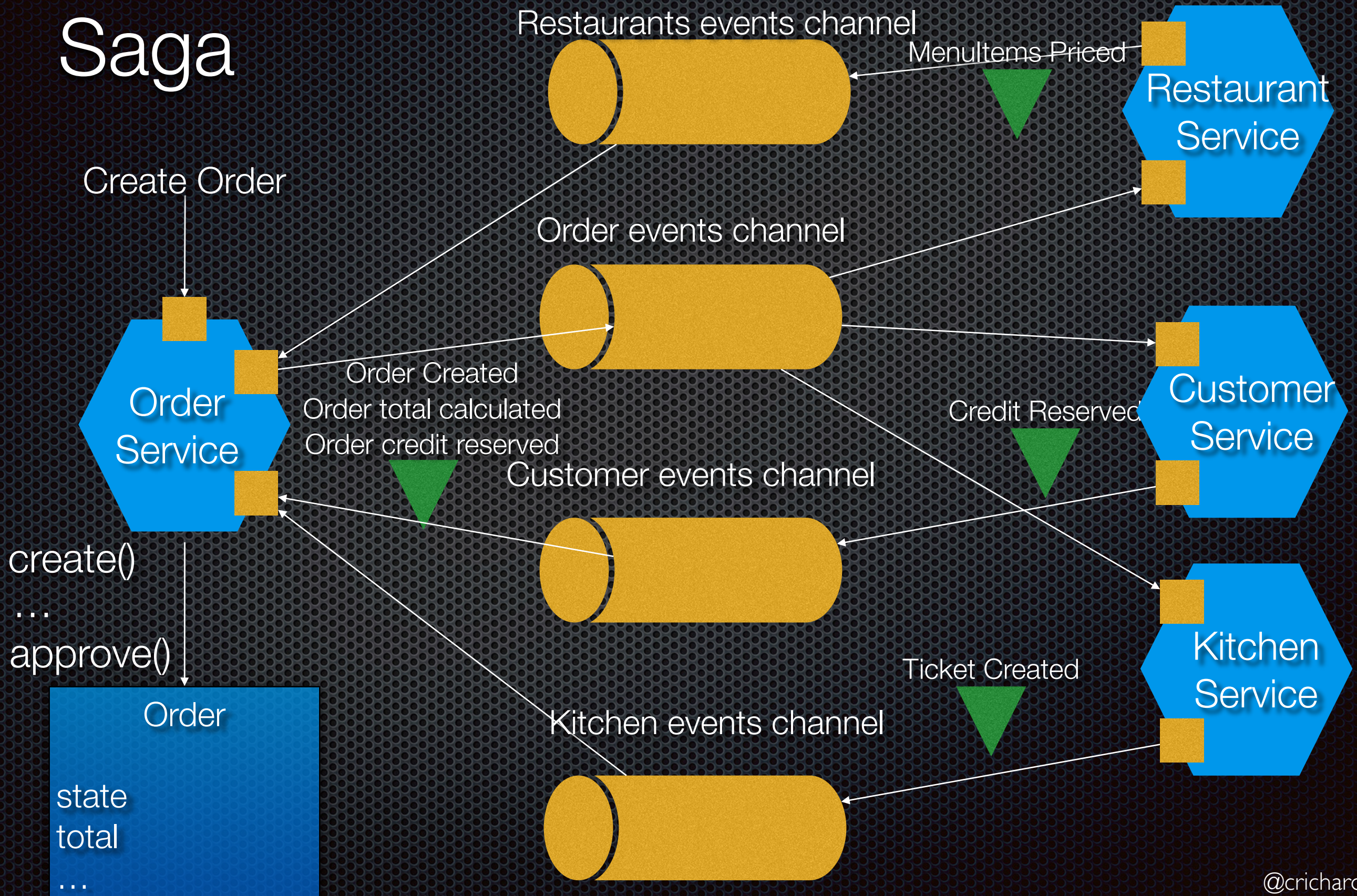
Saga coordination =
sequencing of steps

Choreography: **distributed** decision making

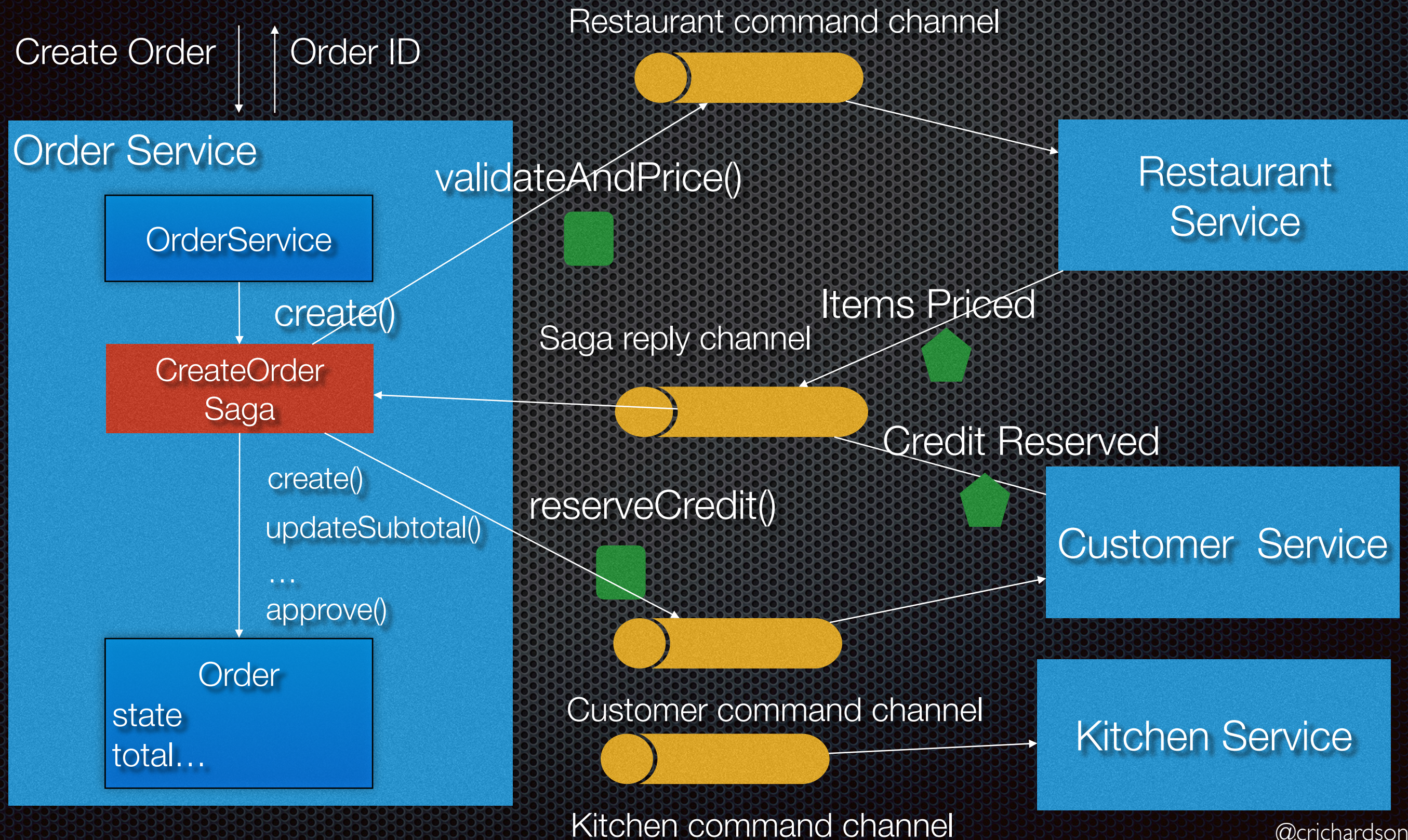
vs.

Orchestration: **centralized** decision making

Choreography-based Create Order Saga



CreateOrderSaga orchestrator



Dark energy repulsive forces

Support multiple technology stacks



Dark matter attractive forces

Simple interactions

? Depends on command

Prefer ACID over BASE



Minimize runtime coupling

✓ Most likely but it depends

Efficient inter-service communication

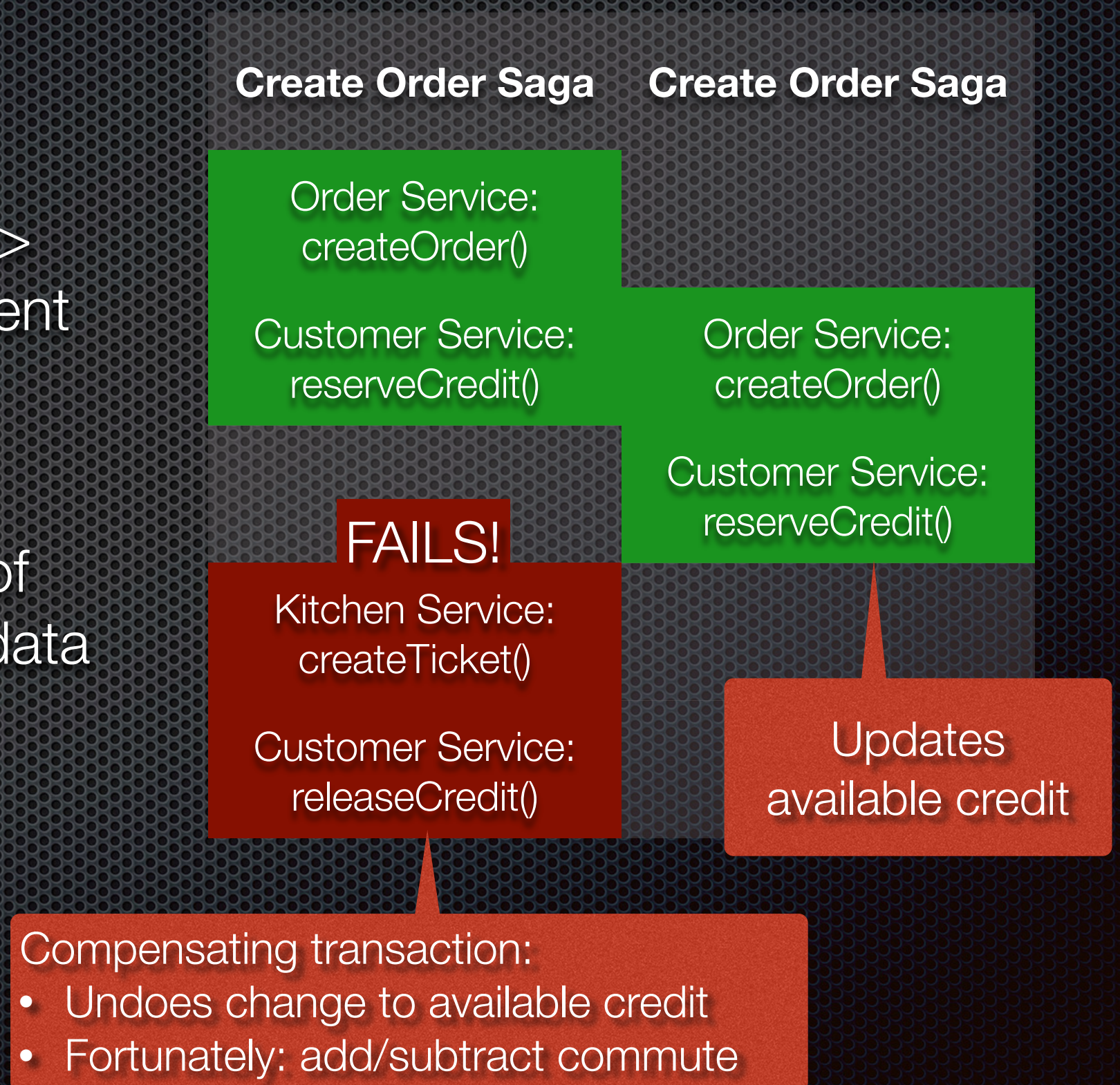
? Depends on command

Minimize design time coupling

? Depends on command

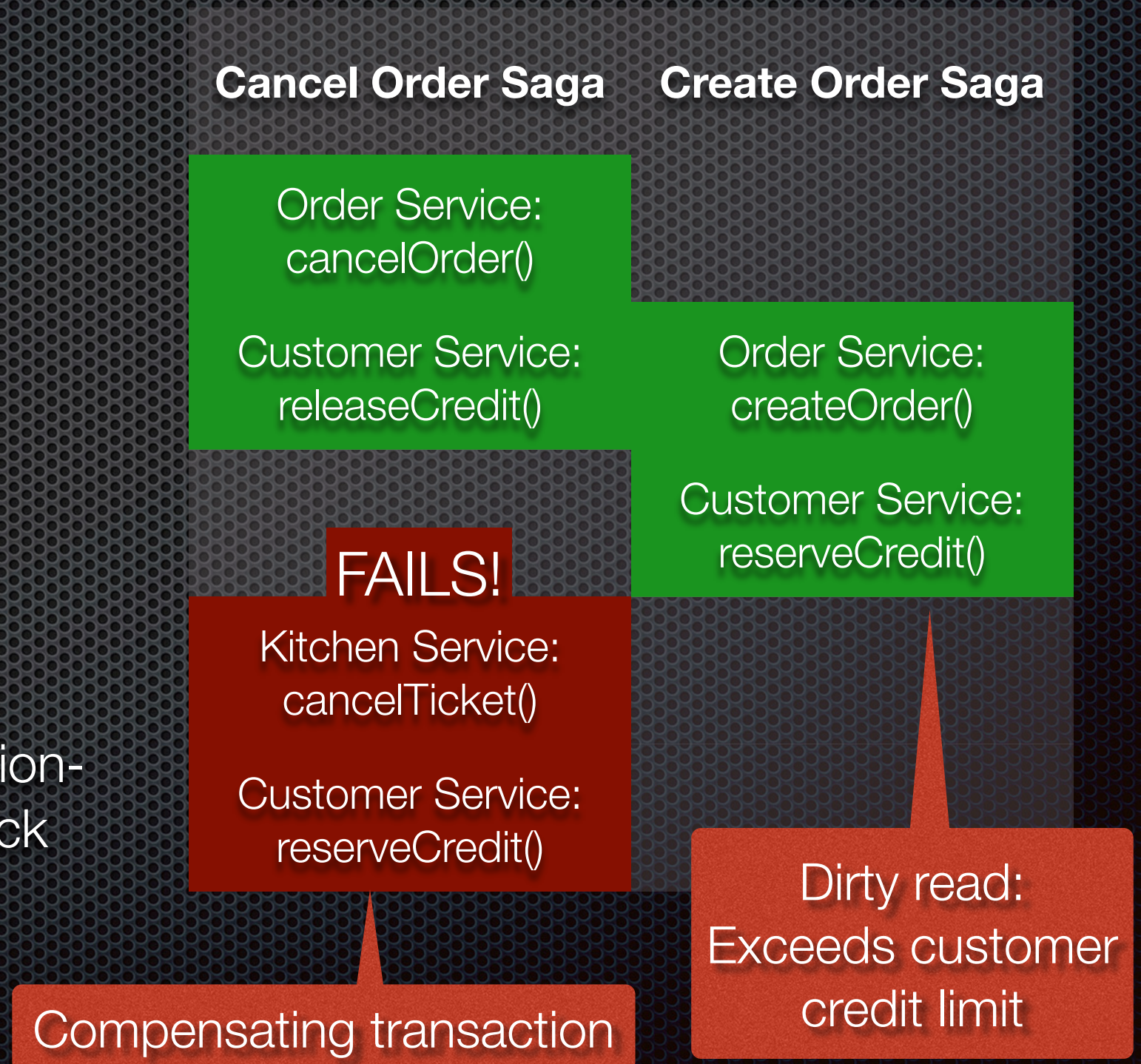
Force: Prefer ACID over BASE

- No automatic undo => extra work to implement compensating transactions
- Potential complexity of undoing changes to data that was changed by another transaction

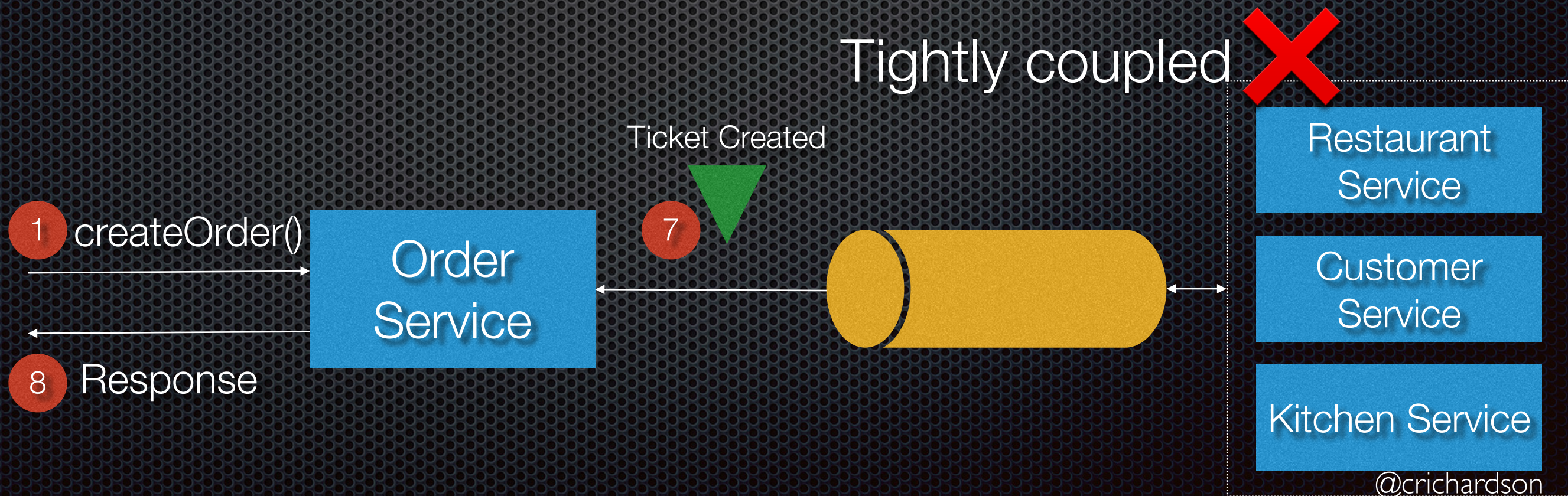
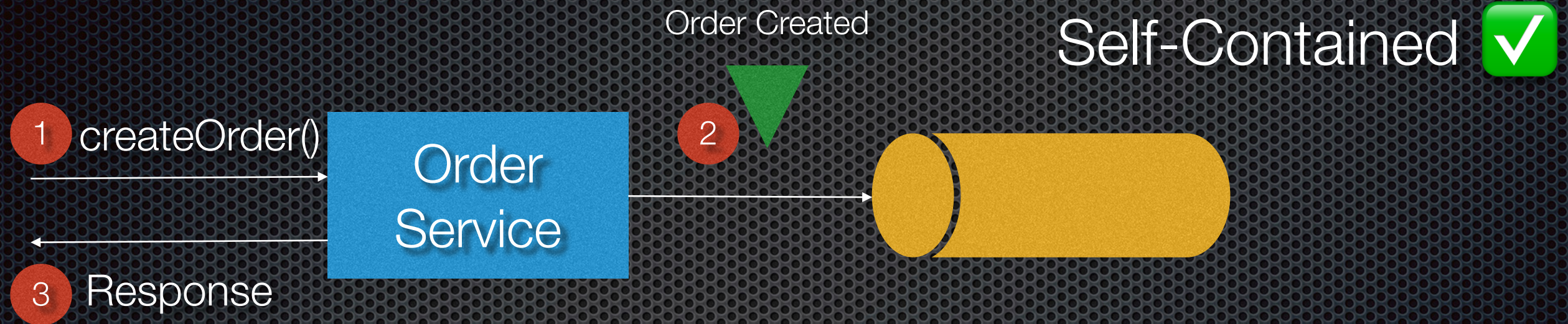


Force: Prefer ACID over BASE

- Sagas are ACD - lack I = isolation => concurrent execution => potential data anomalies:
 - Dirty reads
 - Lost updates
 - Fuzzy reads
- Must implement countermeasures = application-level ACID, e.g. Semantic lock

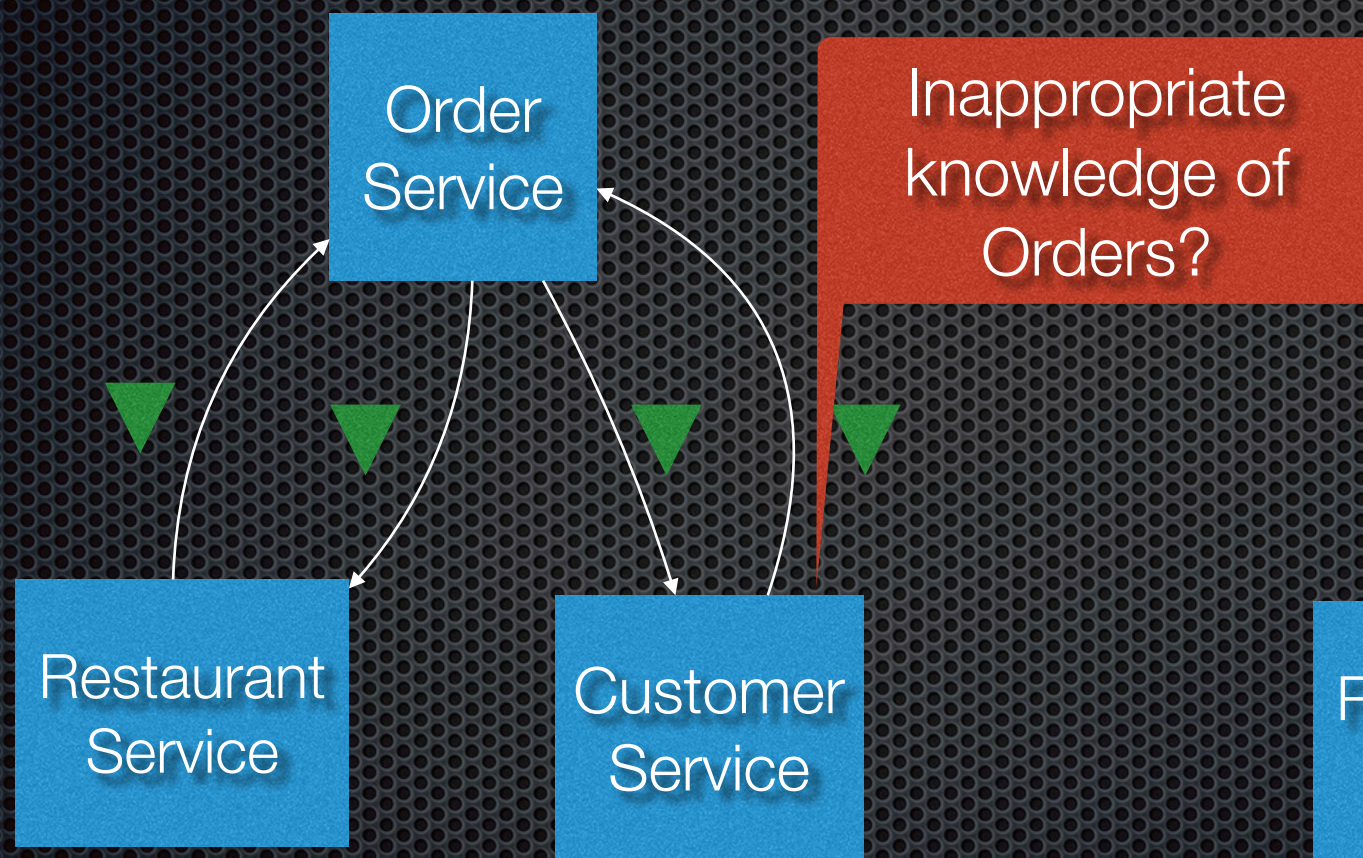


Force: Minimize runtime coupling - depends on when response is sent

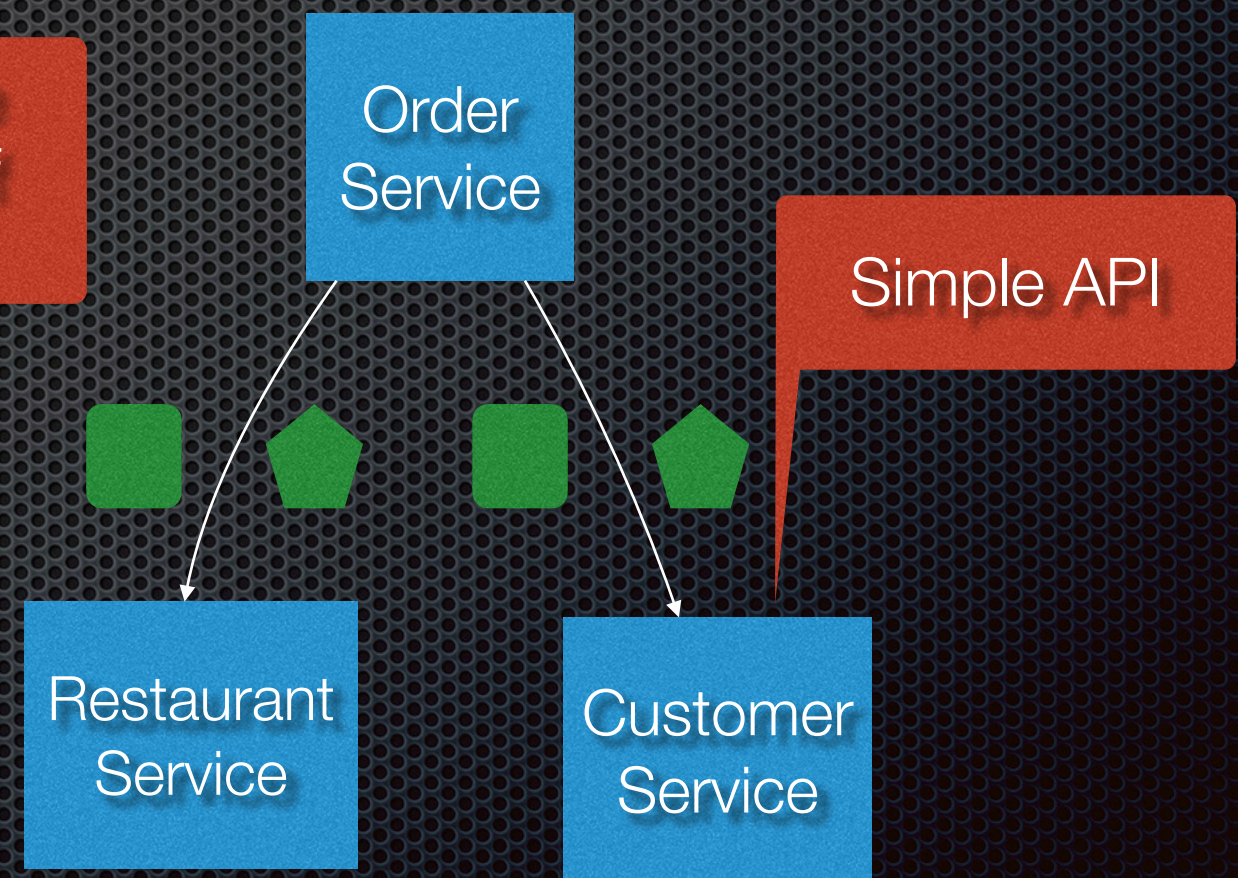


Force: minimize design-time coupling

Choreography



Orchestration



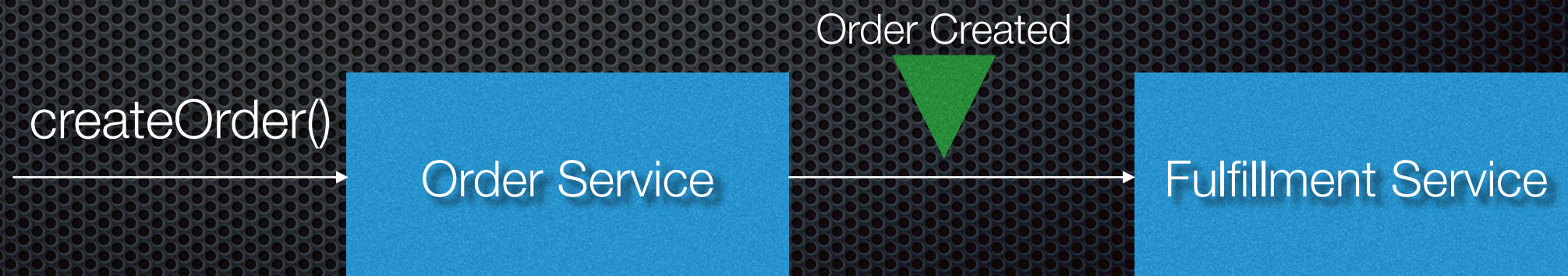
Participants depend on each other Orchestrator depends on participants

Sagas are ***potentially*** complex

BUT

They can be simple

It depends



Don't forget

A saga implements a “request”

It does not involve humans

Documenting a (orchestration-based) saga

	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder()	rejectOrder()
2 (pivot)	Customer Service	reserveCredit()	-
3	Order Service	approveOrder()	-

Updated service definitions: orchestration

- ✦ API Gateway
 - ✦ Route request to Orchestrator service
- ✦ Orchestrator service
 - ✦ New or existing service
 - ✦ Add service command for the system command
- ✦ Other participants
 - ✦ Add service operations that are invoked by orchestrator

Documenting a choreography-based saga

Step	Triggering event	Participant Service	Command	Published event(s)
1	-	Order Service	createOrder()	• Order Created
2 (Pivot)	Order Created	Customer Service	reserveCredit()	• Credit Reserved • Credit Limit Exceeded
3a	Credit Reserved	Order Service	approveOrder()	• Order Approved
3b	Credit Limit Exceeded	Order Service	rejectOrder()	• Order Rejected

Updated service definitions: choreography

- ✦ API Gateway
 - ✦ Route request to Initial participant
- ✦ Initial participant
 - ✦ Add service command for the system command
- ✦ All participants
 - ✦ Add required event handlers
 - ✦ Publish required events

Agenda

- Overview of designing commands
- The Saga pattern
- Designing sagas
- The Command-side replica pattern

The saga design process

1. Define steps - $T_1, T_2, \dots T_i$ (ACID transaction in a participant)
2. Evaluate different orders
 1. Consider ordering constraints
 2. Identify simplest and to possibly implement countermeasures
3. Define each C_i - if and only if later $T_{j>i}$ can fail
4. Select countermeasure if necessary
5. Select choreography or orchestration

Defining saga steps: simple case = one step per service

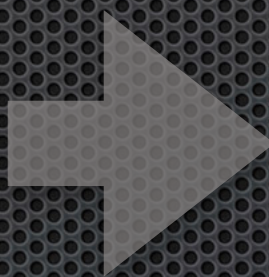
createOrder()

Validate Consumer

Validate and price menu items

Authorize credit card

Create Order



Step	Participant	Operation
1	Consumer Service	validate Consumer()
2	Restaurant Service	validateAnd PriceItems()
3	Credit Card Service	authorize Card()
4	Order Service	create Order()

Defining saga steps: 'complex'

case = multiple steps per service

createOrder()

Get Customer Discount Schedule

Price Order using Discounts

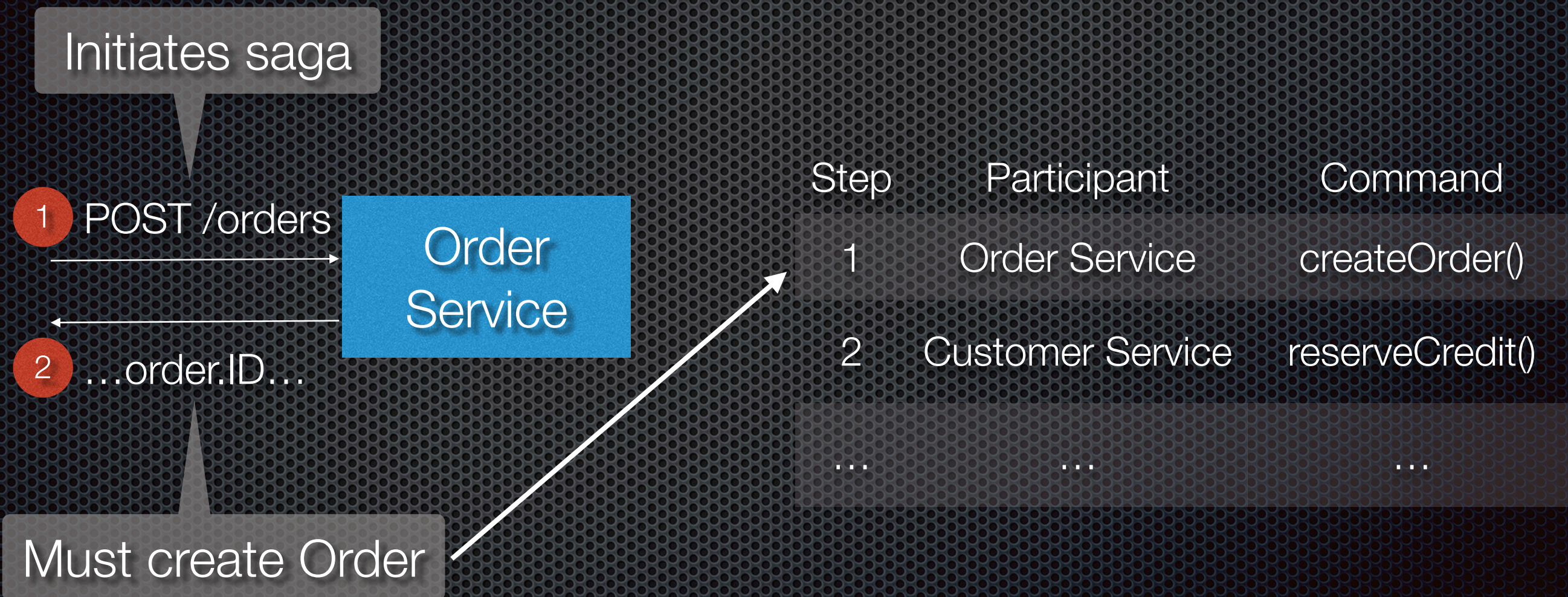
Reserve credit

Create Order

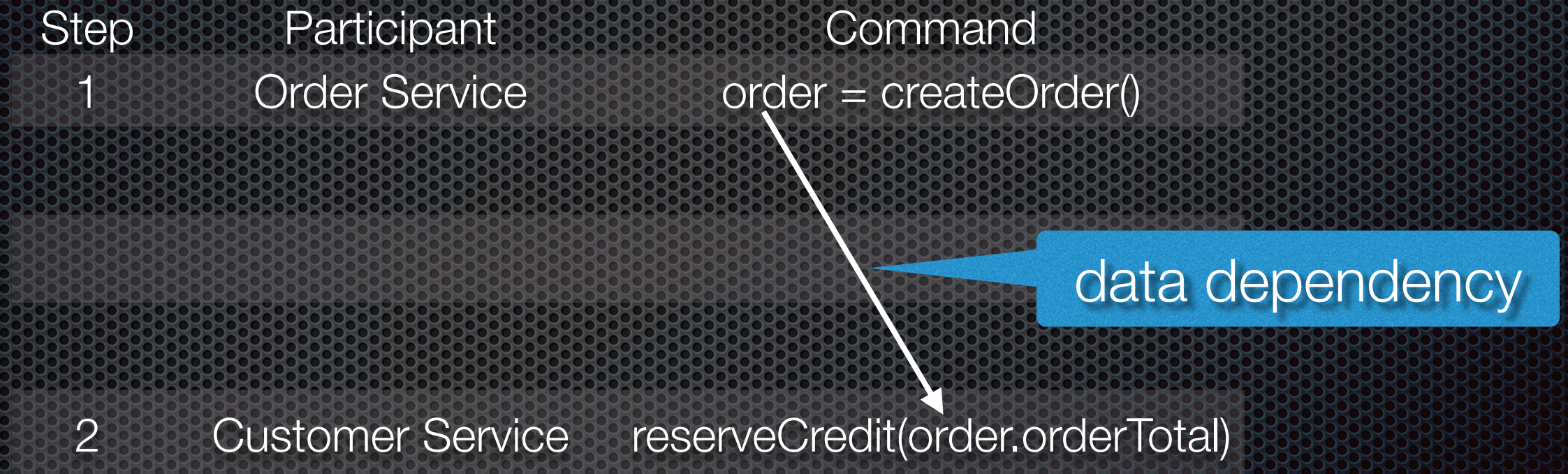


Step	Participant	Operation
1	Customer Service	getDiscountSchedule()
2	Order Service	priceOrder()
3	Customer Service	reserveCredit()
4	Order Service	createOrder()

Criteria for ordering steps: data required by response

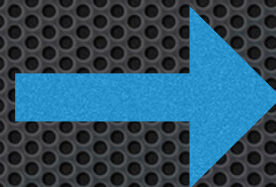


Criteria for ordering steps: data dependencies



Criteria for ordering steps: minimizing compensating transaction complexity

Step	Participant	Command	Compensating transaction
...	Customer Service	UpdateComplexProfile()	complicatedUndo()
...			



Step	Participant	Command	Compensating
...	Customer Service	prepareForUpdate()	simpleUndo()
...			
PIVOT TRANSACTION			
...			-
...	Customer Service	UpdateComplexProfile()	-

Designing compensating transactions

Initiated by OrderService.createOrder()

	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder'()	rejectOrder()
2 (Pivot txn)	Customer Service	reserveCredit()	-

Can fail

Required

Adding a compensating transaction

Customer
Service

reserveCredit()

reserveCredit(customerId, orderTotal)

Pre-conditions:

- Customer.customerId is valid
- orderTotal ≤ Customer.availableCredit

Post-conditions

- Customer.availableCredit -= orderTotal

OrderService

createOrder'()
rejectOrder()

createOrder'(customerId, orderTotal)

Post-conditions

- Order was created
- Order.state = PENDING

rejectOrder(orderId)

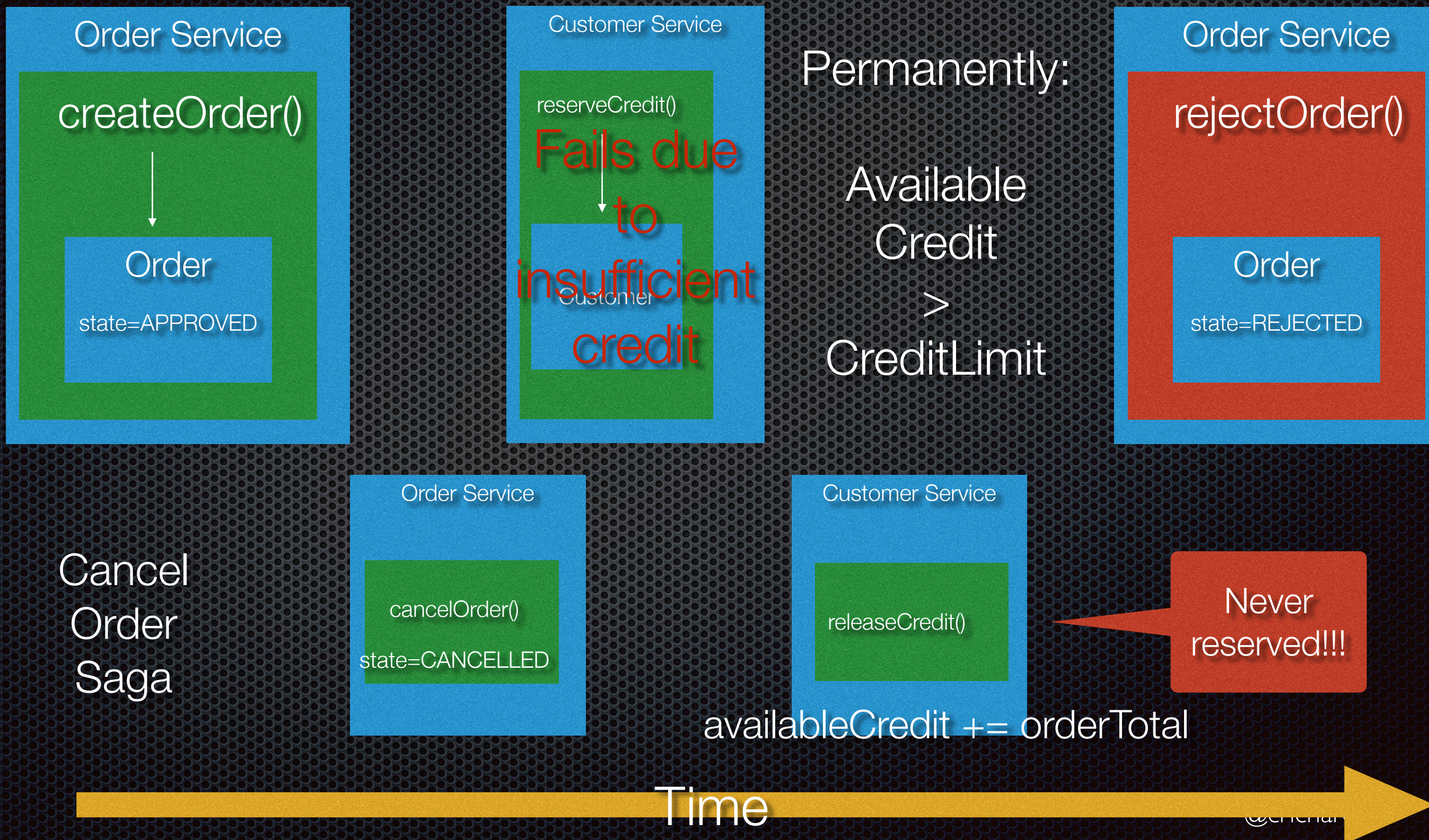
Post-conditions

- Order.state = REJECTED

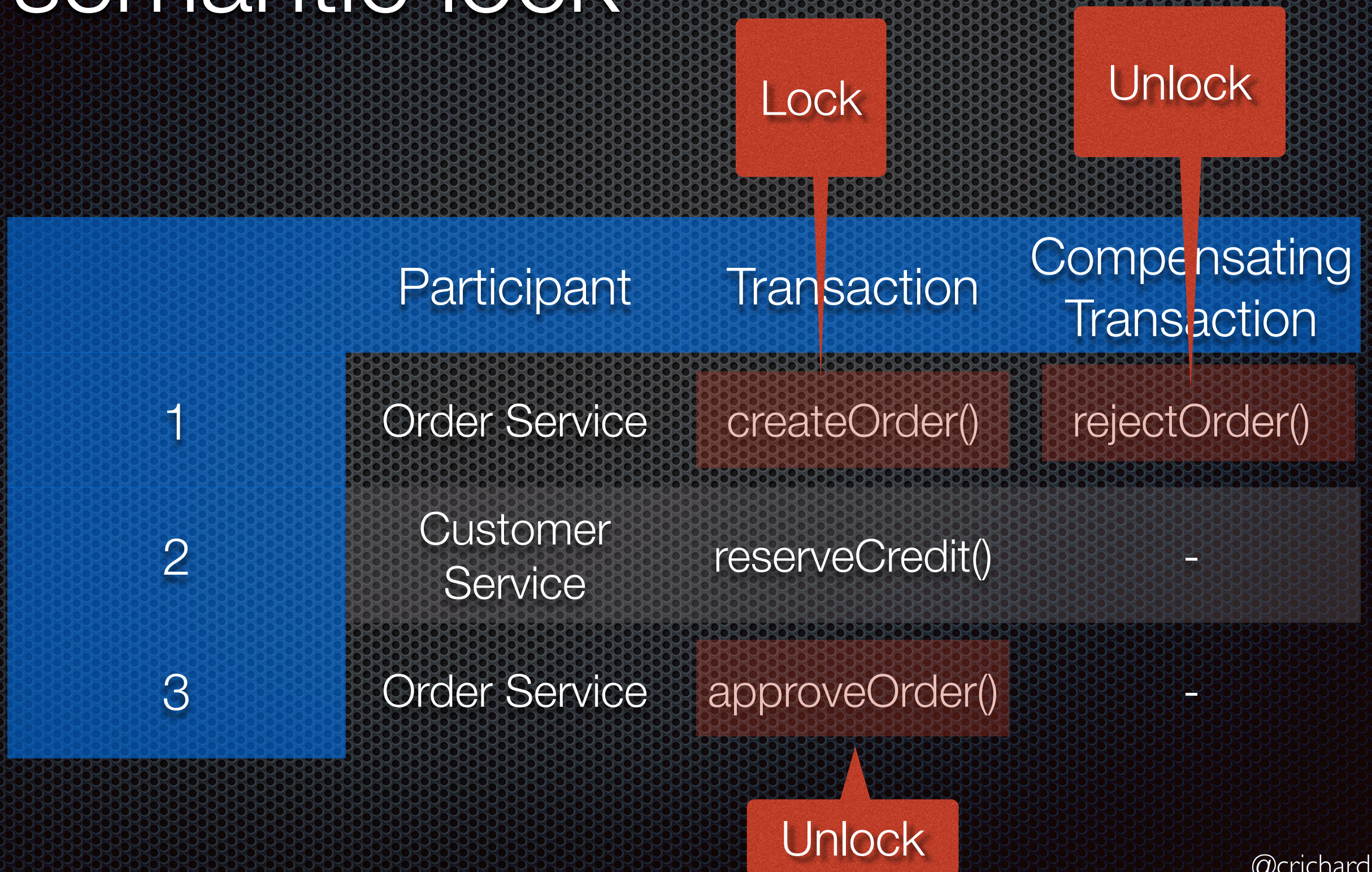
Compensating
transaction

Designing countermeasures - making sagas more ACID

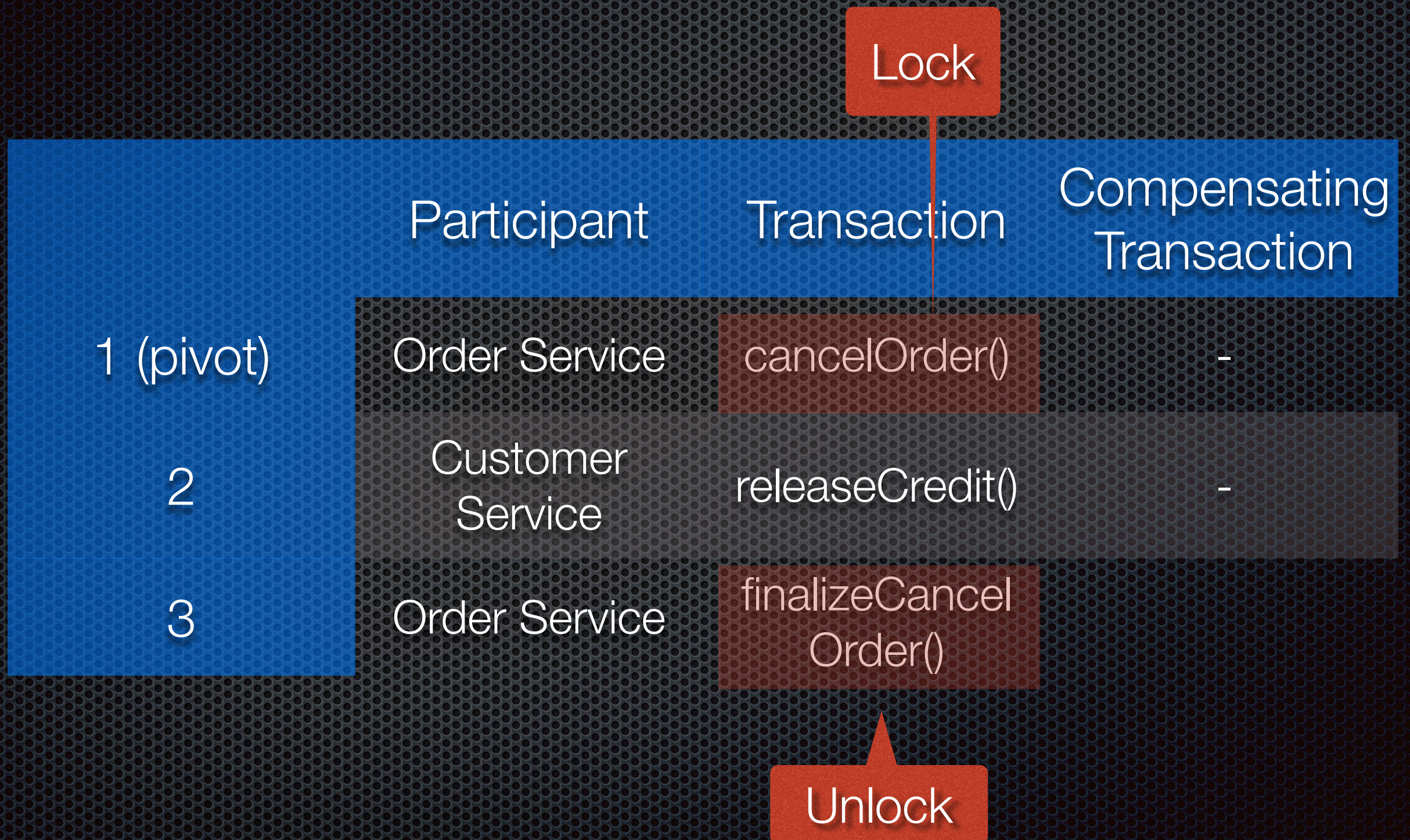
But with `cancelOrder()` => inconsistent data
=> need a countermeasure



Create Order Saga - with semantic lock



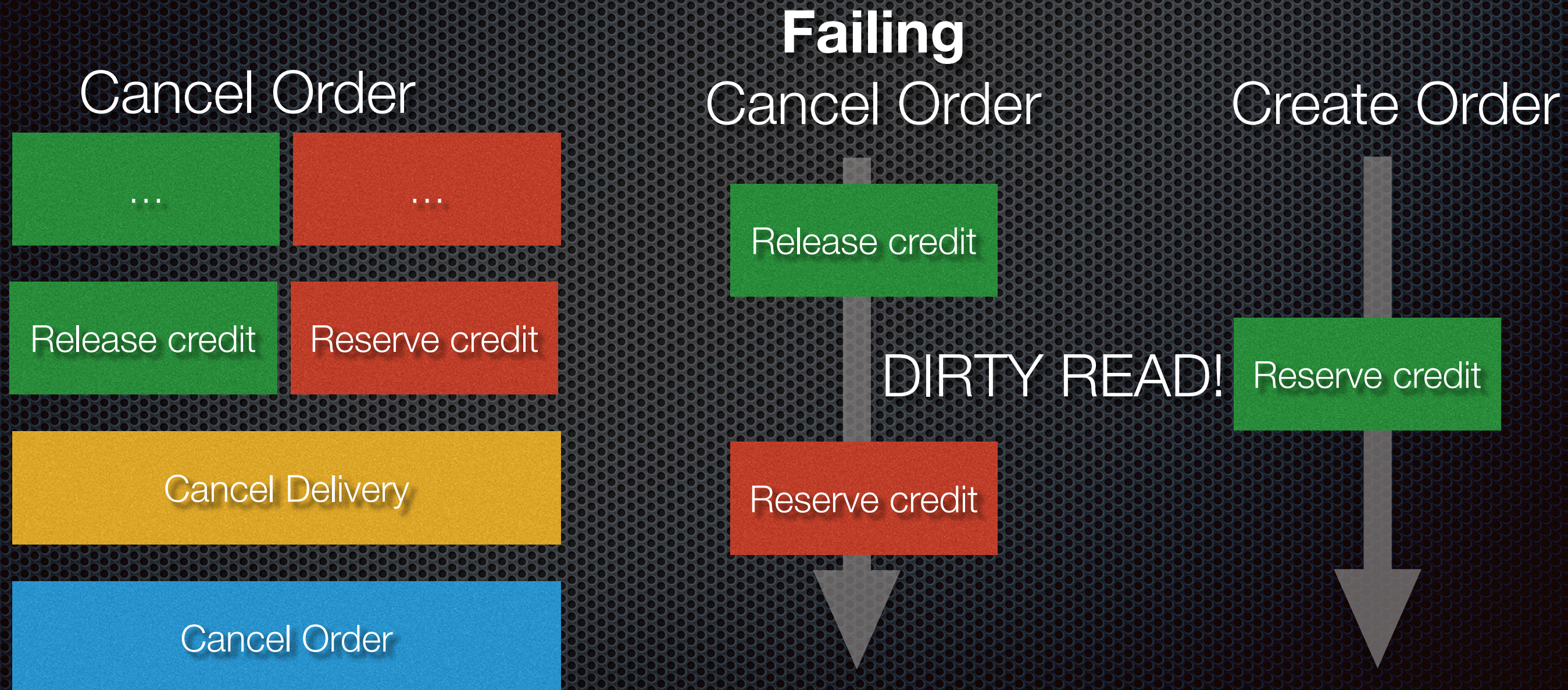
Cancel Order + semantic lock



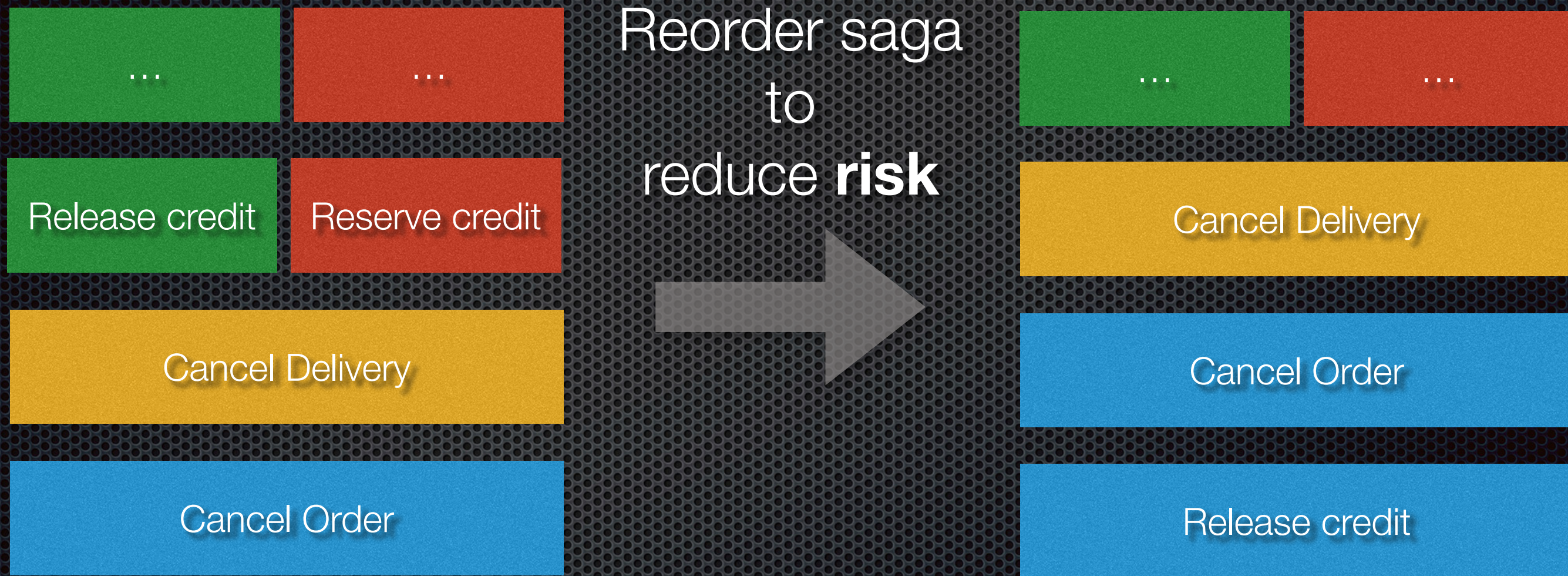
About semantic locks

- A semantic lock is a
 - “flag” in a domain object, e.g. Order
 - Boolean flag, e.g. boolean locked
 - Additional state, e.g. PENDING
 - Entry in separate lock table
- All operations that access the domain object must observe semantic lock, eg. Create Order, Cancel Order, ...
- When an entity is locked:
 - Return a “try again later” error...
 - Wait and retry

Countermeasure: Pessimistic view - avoids dirty reads



Countermeasure: Pessimistic view = reorder steps

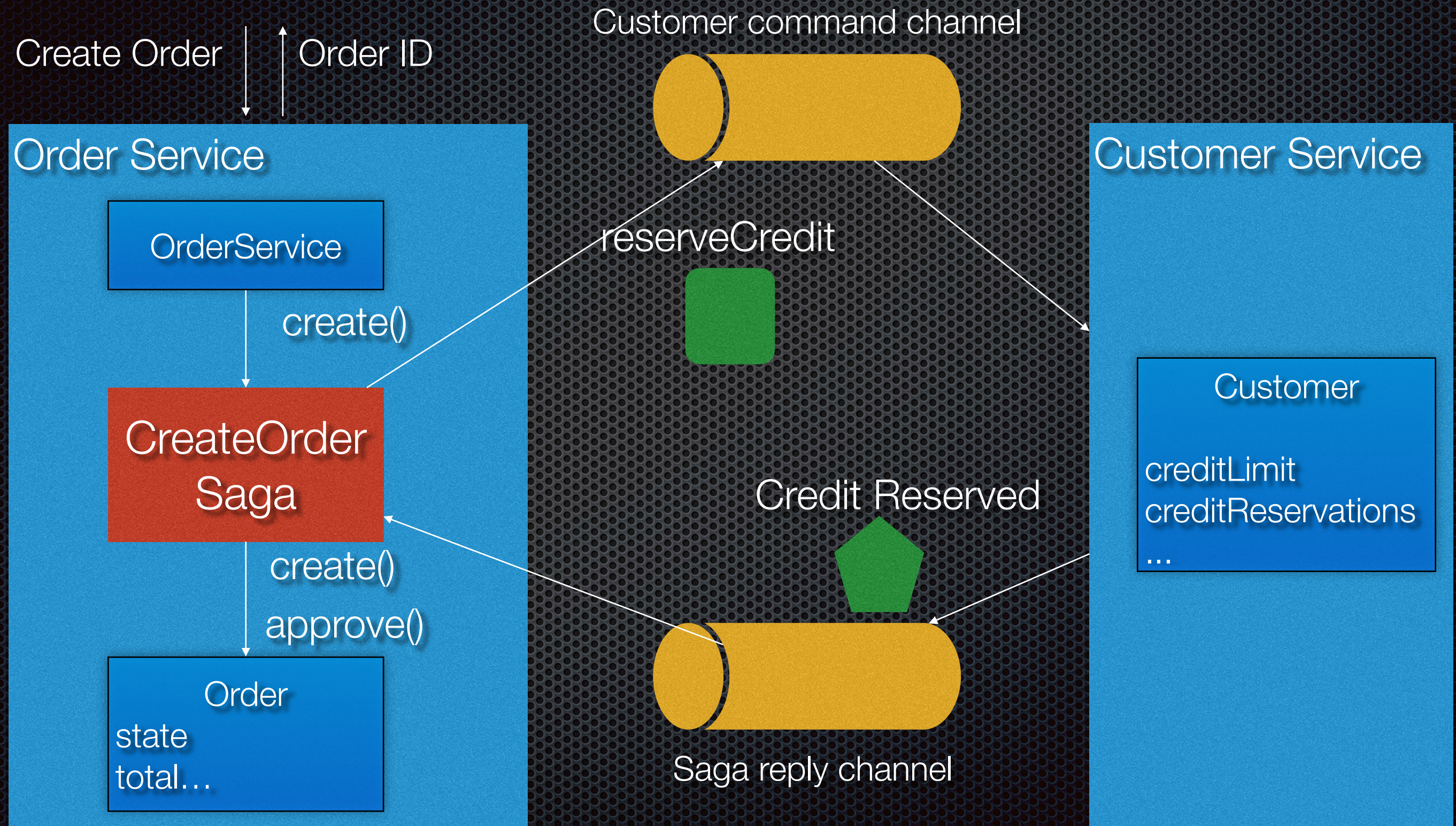


Won't be compensated, $\Rightarrow$ one update $\Rightarrow$ NO dirty read

Designing saga coordination: choreography vs. orchestration

	Choreography	Orchestration
Understandability	Distributed coordination logic => complex sagas can be difficult to understand BUT can be simple	Centralized coordination logic can be easier to understand
Design-time coupling	Potentially problematic cycles: participants depend upon each other	Simpler: Orchestrator depends on participants But the orchestrator is coupled to all participants

Orchestration-based CreateOrderSaga



Designing an orchestration-based saga

- Simply map saga definition to orchestration-based implementation
- Key decisions:
 - Pick orchestration framework
 - Decide location of orchestrator
 - Design commands and replies

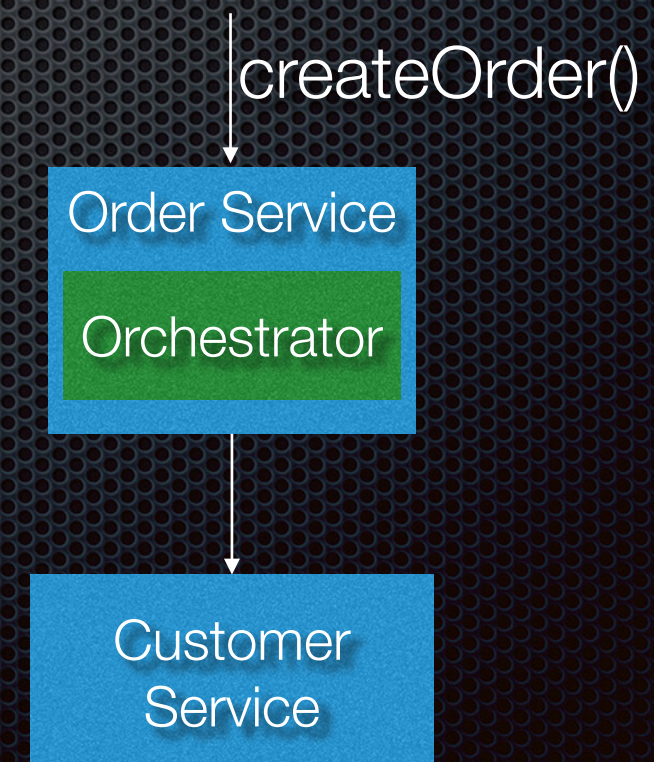
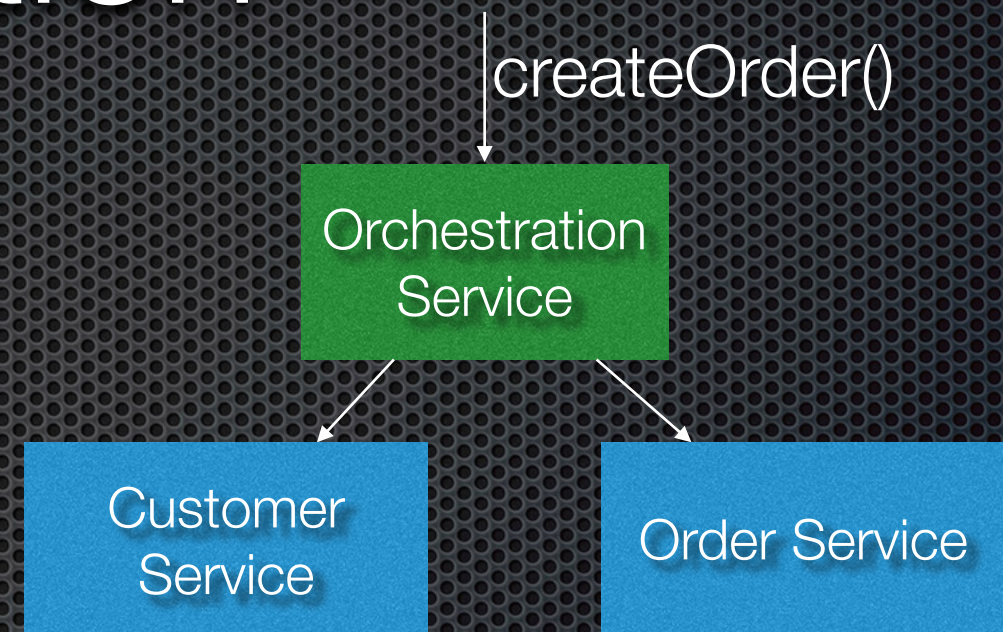
	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder()	rejectOrder()
2 (pivot)	Customer Service	reserveCredit()	-
3	Order Service	approveOrder()	-



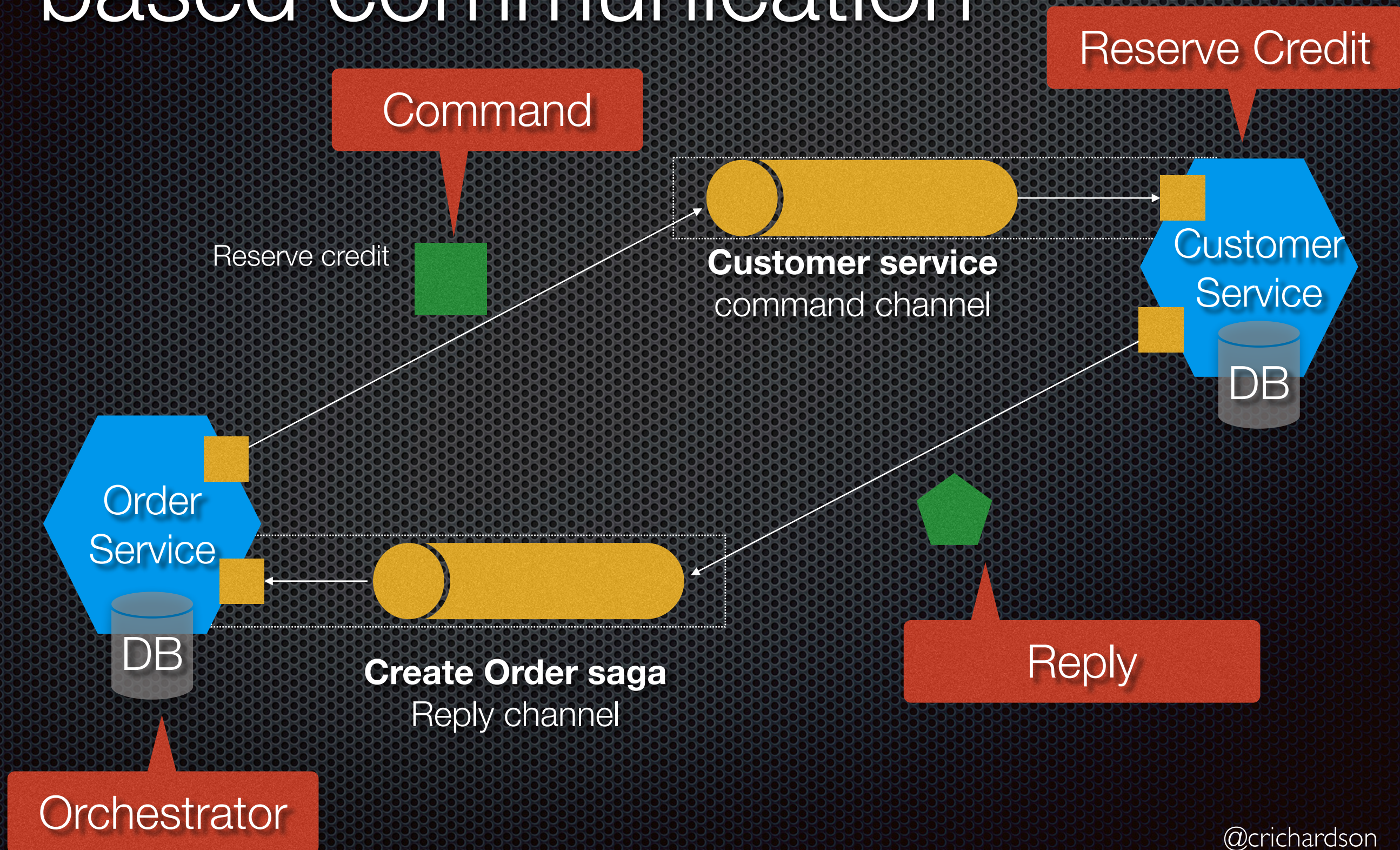
```
public class LocalCreateOrderSaga implements SimpleSaga<LocalCreateOrderSagaData> {  
  
    private DomainEventPublisher domainEventPublisher;  
    private OrderRepository orderRepository;  
  
    public LocalCreateOrderSaga(DomainEventPublisher domainEventPublisher,  
                                OrderRepository orderRepository) {  
        this.domainEventPublisher = domainEventPublisher;  
        this.orderRepository = orderRepository;  
    }  
  
    private SagaDefinition<LocalCreateOrderSagaData> sagaDefinition =  
        step()  
            .invokeLocal(this::create)  
            .withCompensation(this::reject)  
            .step()  
            .invokeParticipant(this::reserveCredit)  
            .step()  
            .invokeLocal(this::approve)  
            .build();  
}
```


Orchestration-based saga: orchestrator location

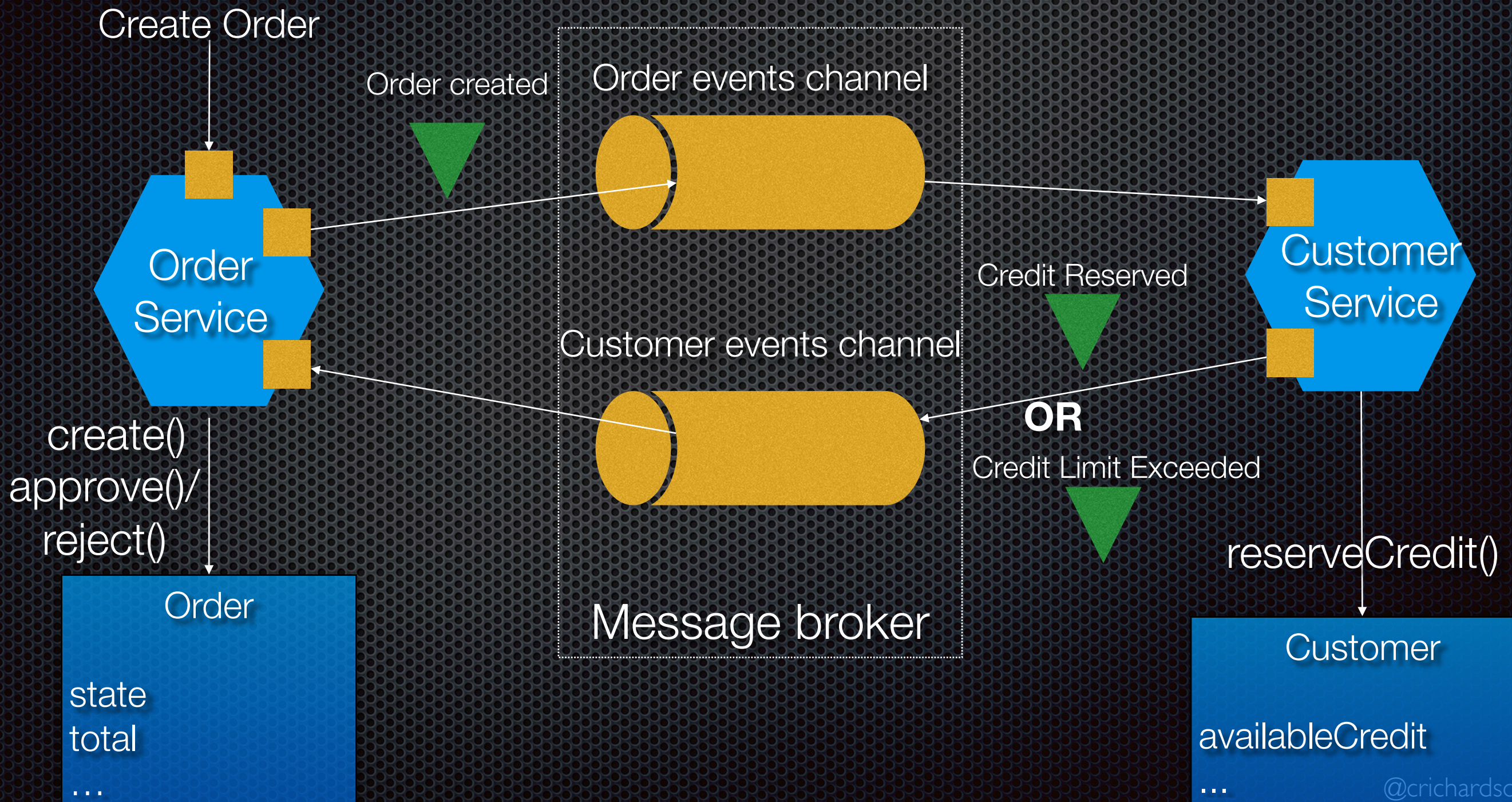
- Issue = design-time coupling
 - Orchestrator -> participants
 - Tramp coupling: Client - participant, Participant - participant
- Options
 - Separate orchestration service
 - Design time coupled to participants
 - Runtime coupled to some (first step) participants
 - Embedded in a participant
 - The participant is coupled to the other participants



Request/asynchronous reply-based communication



Choreography-based Create Order Saga



Designing a choreography-based saga

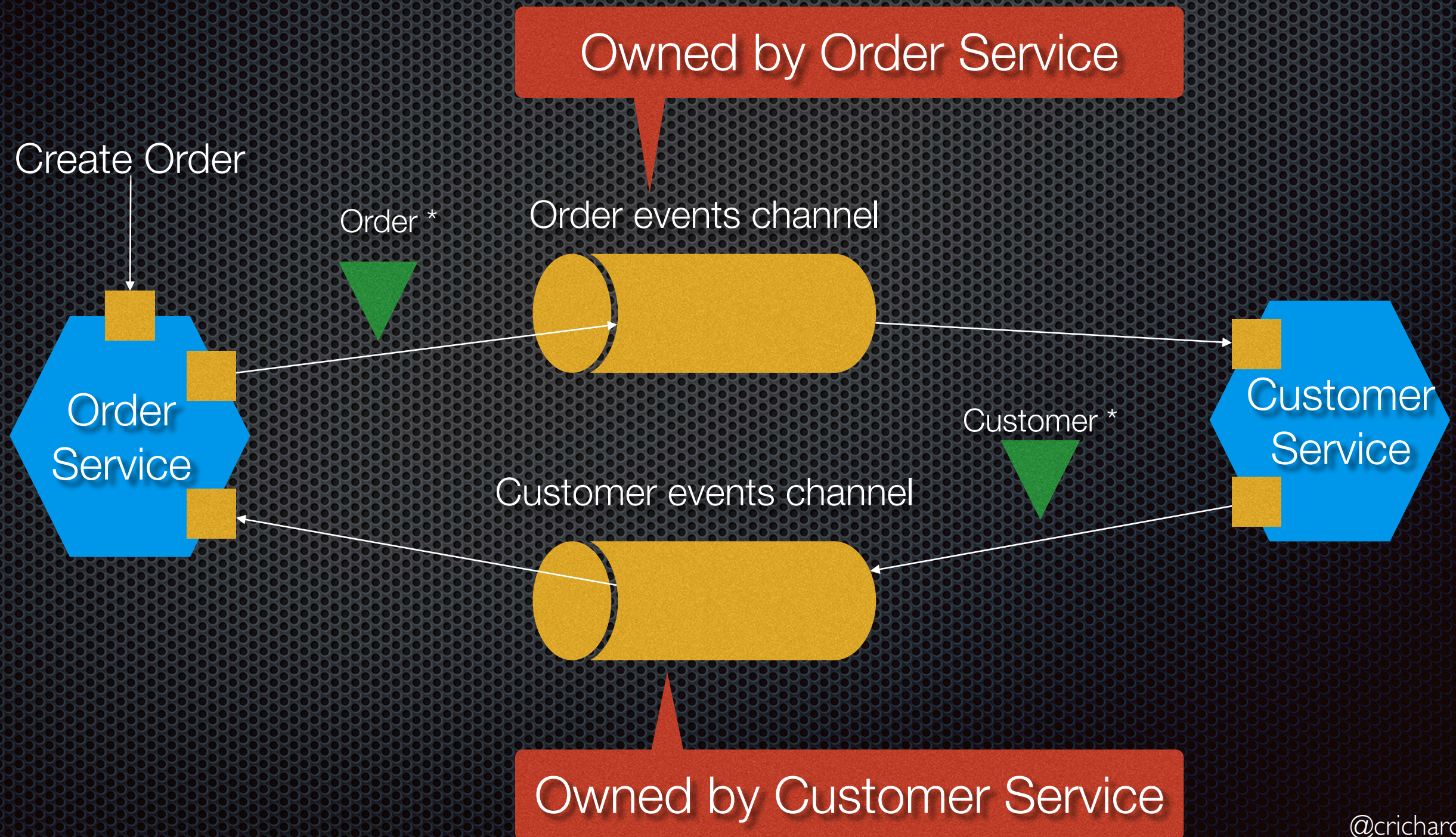
- Design events that each step publishes
- Step N must publish
 - success(s?) event to trigger step T(N + 1)
 - failure event(s) to trigger step C(N - 1) (or earlier)

	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder()	rejectOrder()
2 (pivot)	Customer Service	reserveCredit()	-
3	Order Service	approveOrder()	-



Step	Triggering event	Participant Service	Command	Published event(s)
1	-	Order Service	createOrder()	• Order Created
2	Order Created	Customer Service	reserveCredit()	• Credit Reserved • Credit Limit Exceeded
3a	Credit Reserved	Order Service	approveOrder()	• Order Approved
3b	Credit Limit Exceeded	Order Service	rejectOrder()	• Order Rejected

Message channel design

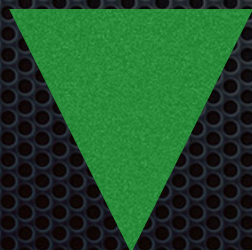


Designing events

Minimal

- Just the essential data
- + Stable event structure
- Consumer needs to (synchronously) fetch the data that it needs = runtime coupling
- Risk of inconsistencies

Order Created

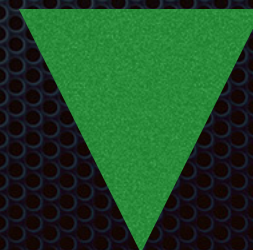


orderId

Enriched

- Contains data needed by a typical consumer, a.k.a. Event-Carried State Transfer
- + Consumer has what it needs
- + Consistent data
- Risk of unstable event structure
- Storage/network overhead

Order Created



orderId
customerId
orderTotal

Agenda

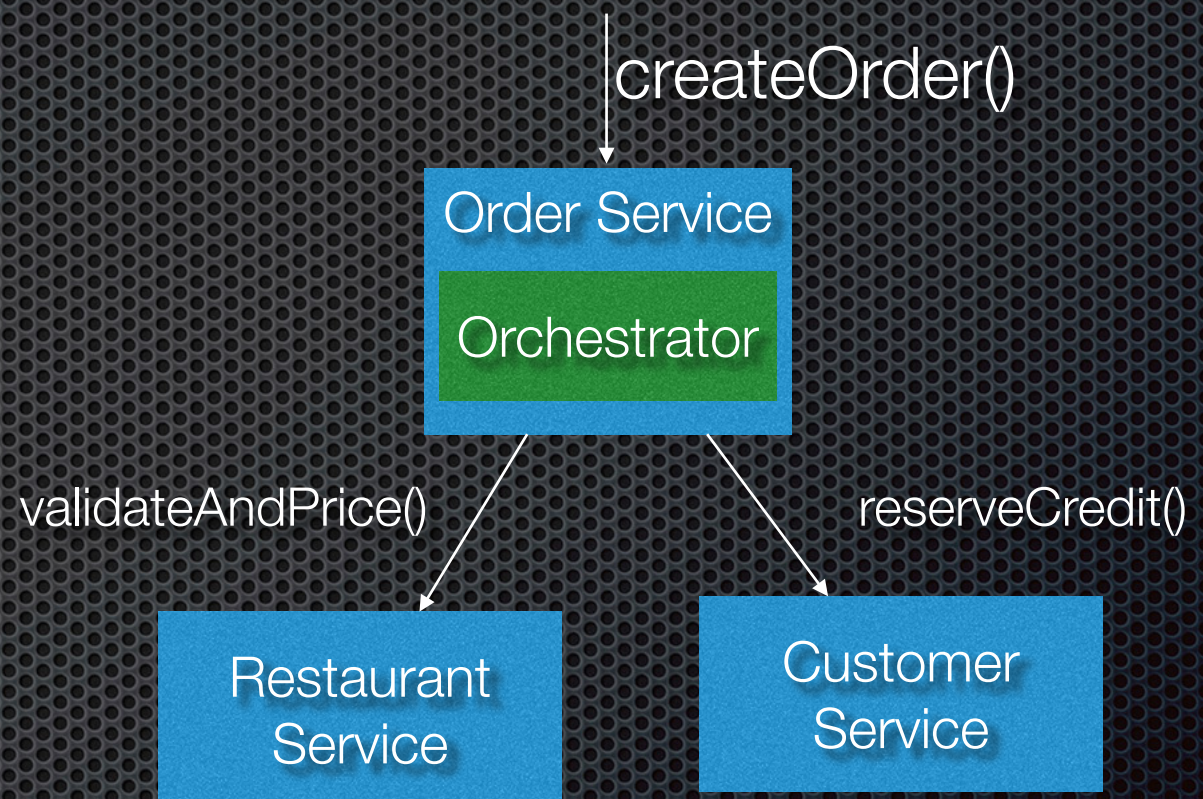
- Overview of designing commands
- The Saga pattern
- Designing sagas
- The Command-side replica pattern

Create Order Saga step: validate and price Menuitems

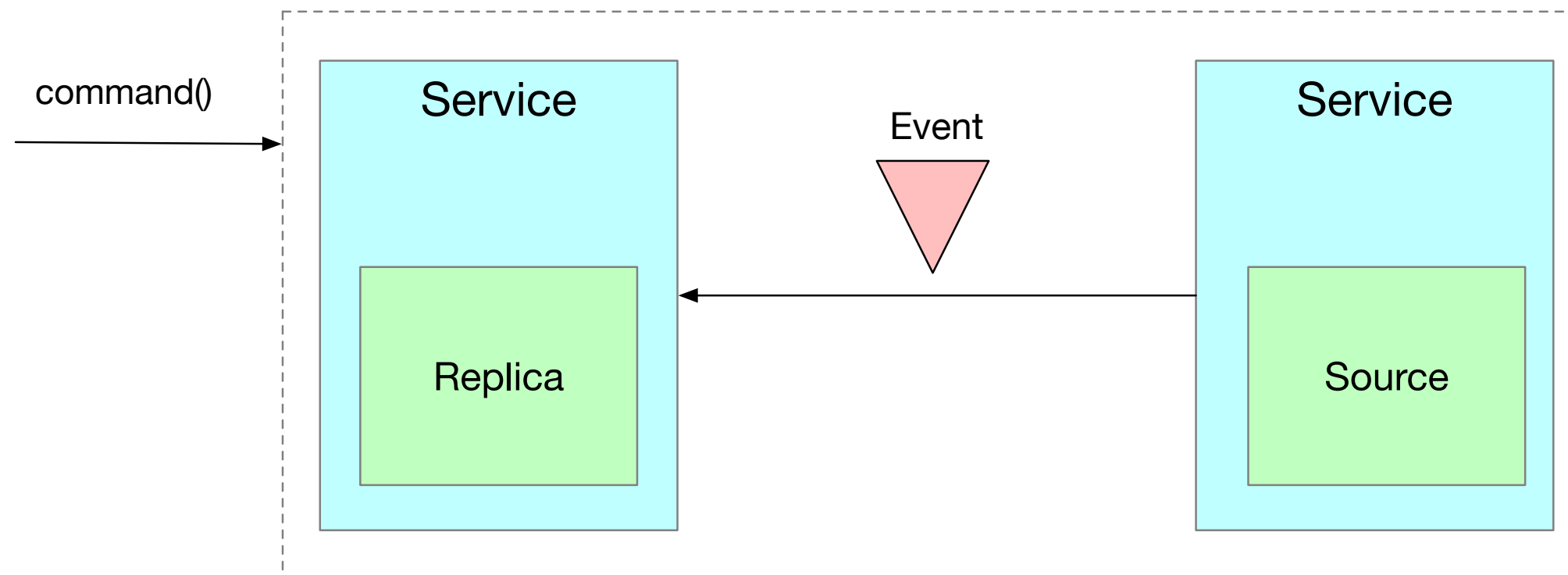
- Issues:
 - Additional saga step
 - HTTP response cannot contain result of validation without runtime coupling
- But Restaurant menus are relatively static

Therefore

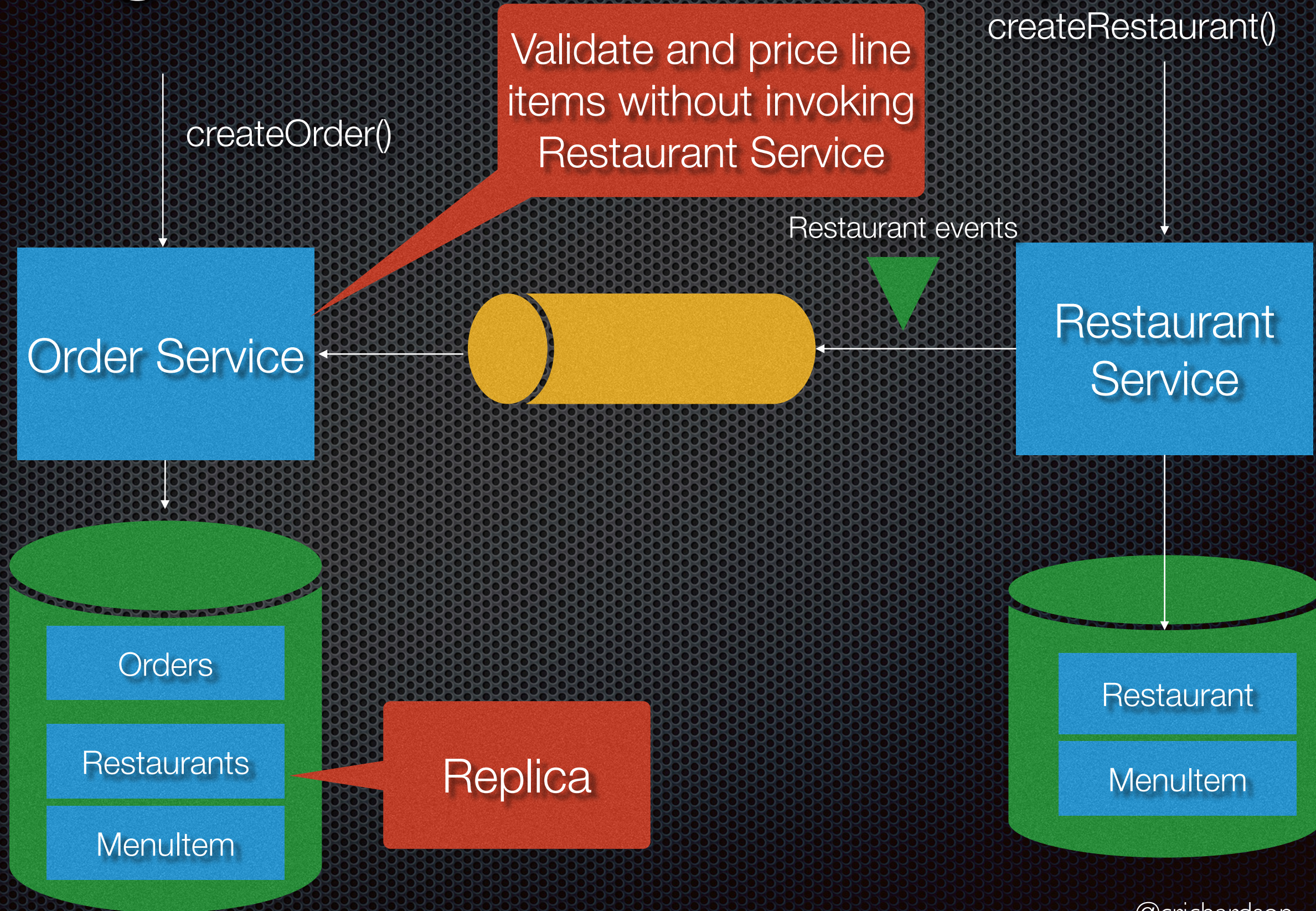
Consider replicating menus to the Order service



Command-side replica



Using a command-side replica



Benefits and drawbacks of Command-side replica

Dark energy repulsive forces

Support multiple technology stacks  e.g. even different DBs

Dark matter attractive forces

Simple interactions

 Eliminates interaction with participant
 Other cmds must publish events

Prefer ACID over BASE

 Risk of stale/inconsistent data

Minimize runtime coupling

 Command does not interact with participant

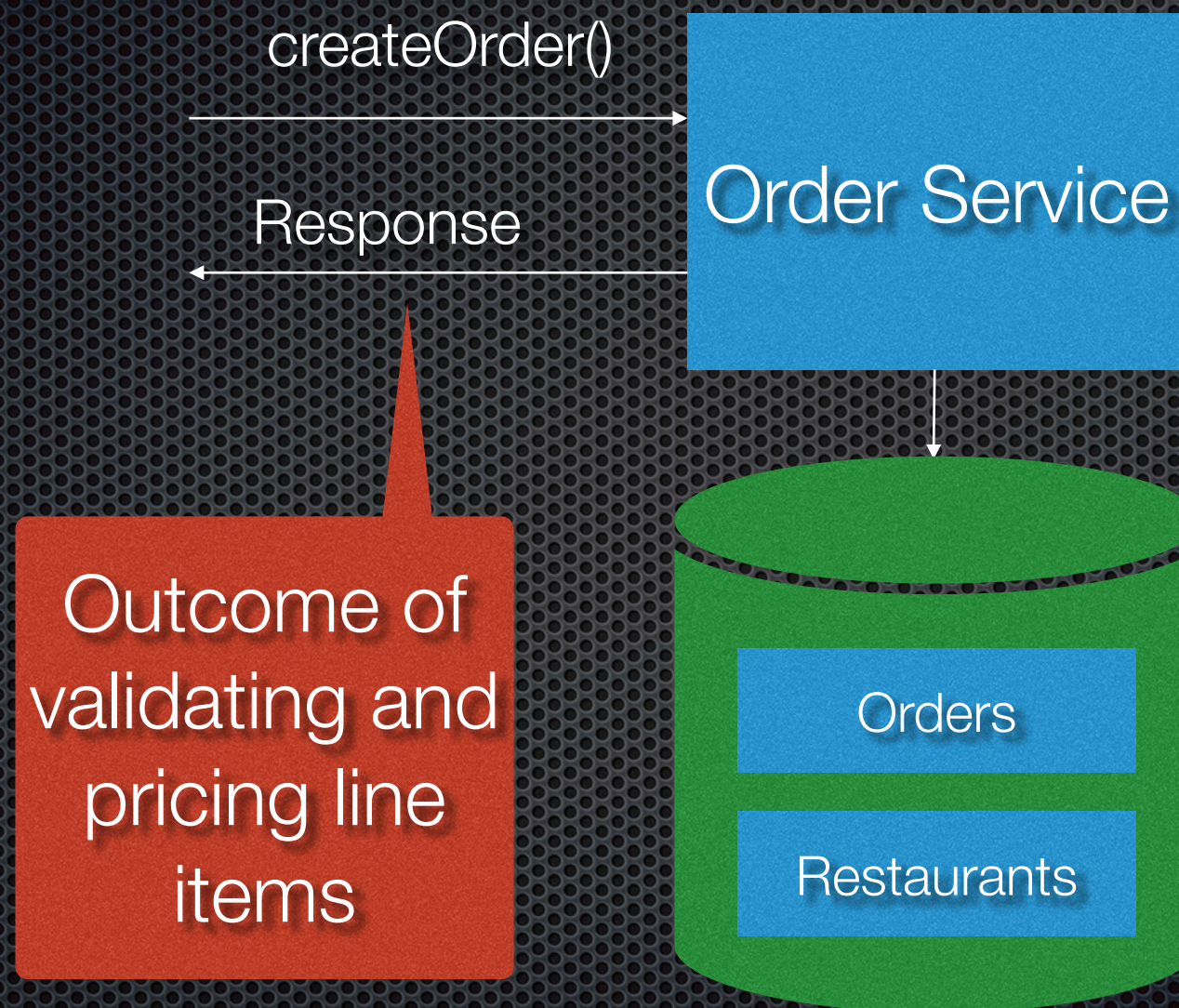
Efficient inter-service communication

 Command is less distributed
 Cost of replication - storage + events

Minimize design time coupling

? Risk of tightly coupling view to providers

Force: Minimize runtime coupling - non-local but self-contained operation

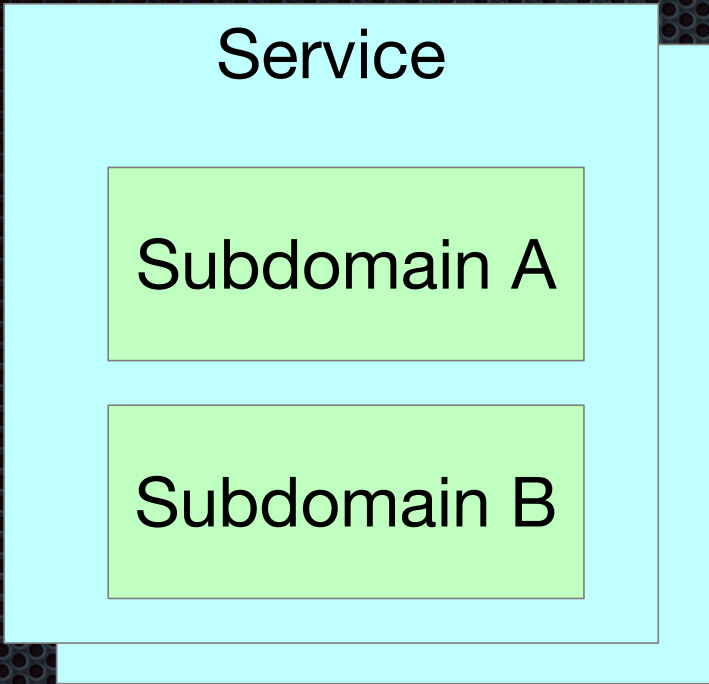


<https://microservices.io/patterns/decomposition/self-contained-service.html>

Updated service definitions: Command-side replica

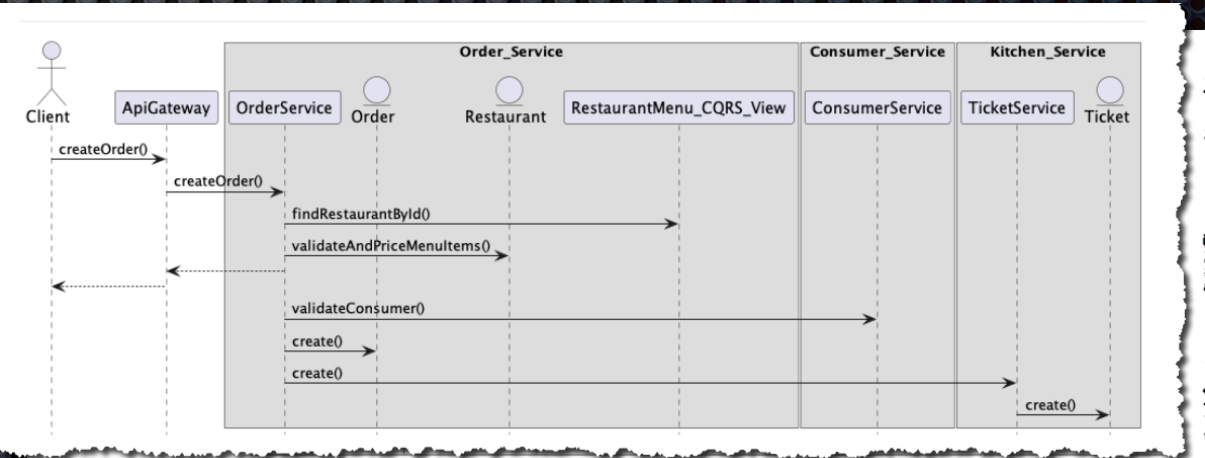
- Replica service
 - Implement event handlers
- Provider services
 - Publish required events

Summary: Deliverables



System command design canvas	
Name:	createOrder()
Entry point:	
Service:	Order Service
Operation:	createOrder()
Collaboration patterns:	
Sagas	• Create Order
Command-side Replicas	• Restaurant Menu View

Service canvas		
Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant ...	



Templates:

- 03-01-ranking-system-operations
- 03-02-designing-operations-and-services

 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

<http://adopt.microservices.io>

About

Kata #3: Rank system operations

Rank the system operations using the template in 03-01-ranking-system-operations

- Rank system operations by importance

Templates:

- 03-01-ranking-system-operations
- 03-02-designing-operations-and-services

About

Chapter 6. Kata #4: Design system commands

Design the services and commands and document them using the templates in [03-02-designing-operations-and-services](#)

- ✦ As a group:
 - ✦ For each command:
 - ✦ Create candidate operation realizations
 - ✦ Evaluate using dark energy/matter
 - ✦ Select the best candidates
- ✦ In pairs: create sequence diagram for assigned commands